

One Forest for a Family: Randomized Weight Functions for Batching $\mathbb{F}_2$ -Linear Integer-MLE Claims over a Characteristic-2 Commitment (the mod- q RLC claim-family construction for F2Z)

Albert Garreta*

2026-07-26

Abstract

F2Z proves a mod- q integer multilinear evaluation $\text{MLE}[\text{INT}(D)](r) = c \in \mathbb{F}_q$ for bit-data committed over a characteristic-2 code, by folding the row variables in the exponent of $\mathbb{K} = \text{GF}(2^{128})$ under a lazy GKR product forest. The forest is 84–96% of the prover, and today each claim pays its own forest — even when the claimed vectors are $\mathbb{F}_2$ -linearly *dependent*, e.g. the triple $(a_1, a_2, a_1 \oplus a_2)$, because $\text{INT}(a \oplus b) = \text{INT}(a) + \text{INT}(b) - 2\text{INT}(a \wedge b)$: the carry term is genuinely new information, and integer-coefficient random linear combinations spend their soundness bits out of the exponent’s 127-bit budget.

This note develops and verifies a batching construction that proves k claims on $\mathbb{F}_2$ -linear combinations of j committed bit-columns, at a shared column point, with **one** forest: the verifier’s random $\mathbb{F}_q$ -coefficients are pushed *into the weight function* and reduced mod q *before* lifting, so the combined weights cost zero extra exponent bits; the forest’s leaves become a public 2^j -case select on the j committed bits (a subset zeta-transform makes them multilinear); and the carry information surfaces as the $2^j - 1 - j$ monomial channels of the pre-sumcheck, discharged by a single degree- $(j+1)$ sumcheck in the cheap field. Soundness costs $1/q$ (tighter than the k/q working estimate) plus standard sumcheck terms. For the XOR triple the ex-ante estimate was ~ 1.3 – $1.8\times$ a single claim against $3\times$ today, with one sent fold vector instead of three; the same-day prototype *confirms* it, measuring 1.74 – $1.87\times$ at $n = 24$ – 28 against 2.5 – $3.7\times$ for the deployed batched path and $\sim 3\times$ for independent proofs, with 26–65% smaller proofs (§8.1). We also delimit the construction: chunks do not collapse (a bandwidth conservation law), fully distinct evaluation points do not batch, and the carry channels are conserved — the construction relocates them from the exponent to the cheap field; it does not eliminate them. One completeness refinement emerged in practice: presum channels whose zeta coefficients

*The base scheme extended here is the one-round characteristic-2 Brakedown instantiation of the char-2 field-switch note (§9 of [2]), a construction due to Lev Soukhanov (levs57) [1], as deployed in the **f2z-pcs** repository [3, 4]. The family-batching construction below was derived in a working session on 2026-07-26 and was *unvetted* at the time of writing; this note is the verification pass. Every identity and bound has been re-derived from scratch, and one working estimate is corrected (Remark 5.3). No soundness gap was found; everything that remains unproven or unmeasured is listed explicitly in §10. A companion prototyping plan lives at `docs/rlc-family-proto-prompt.md`. *Revision (same day)*: the prototype now exists (`prove/verify_mle_eval_mod_q_ligerito_rlc_family`, commits `8fbd155–b0d13f7`) and **confirms the predicted cost band**: the measured XOR triple proves at 1.74 – $1.87\times$ one claim at $n = 24$ – 28 against 2.5 – $3.7\times$ for the deployed baselines (§8.1). Numbers marked (est.) are the ex-ante model, kept as the record the measurement is judged against. The prototype also surfaced one *completeness* refinement the original spec needs: identically-vanishing presum channels must be elided (Proposition 6.4).

vanish identically — which happens for legitimate degenerate families, e.g. every claim on one XOR combination — must be *elided* by both parties (Proposition 6.4).

Contents

1	Introduction	3
1.1	The problem	3
1.2	The construction, in one paragraph	3
1.3	Results	4
1.4	Status, provenance, attribution	5
2	The base scheme and the interfaces we use	5
2.1	Data, statement, notation	5
2.2	The pipeline	5
2.3	The two lemmas the construction stands on	6
3	Why there is no naive discount	7
3.1	Carries: XOR claims are genuinely new information	7
3.2	Integer-coefficient RLCs spend the exponent budget	8
3.3	The door: reduction mod q is semantically free	8
4	The construction	9
4.1	Statement, ordering, and the case-collapsed weight function	9
4.2	One forest: leaves as a 2^j -case select	9
4.3	The multi-channel presum	10
4.4	Why a separate n -variable discharge is forced	11
4.5	The monomial discharge	12
4.6	The verifier	12
5	Security	13
5.1	Completeness	13
5.2	The γ -combination	14
5.3	What changed, and what is inherited	14
5.4	Soundness	14
6	Special cases	16
6.1	Multi-point batching: one column, many row points	16
6.2	Affine forms, complements, and <code>complement_elision</code>	16
6.3	Vanishing channels and channel elision	17
7	Boundaries and negative results	18
7.1	Chunks do not collapse: a conservation law	18
7.2	Fully different points do not batch	18
7.3	Carries are conserved	19
7.4	Case tables scale 2^j	19
8	Cost model	19
8.1	Measured: the prototype confirms the band	20

9	Structured taps: index-translated operands	22
9.1	Setting and the stream reduction	22
9.2	Openings: translated eq is a carry matrix product	22
9.3	Measured: the stream family loses to extraction	23
9.4	The collapse: uniform-op claims are weight transforms	24
9.5	The composed collapse: uniform ops over mixed sources	25
10	Open questions	25

1 Introduction

1.1 The problem

The F2Z polynomial commitment scheme [3, 4] realises §9 of the characteristic-2 field-switch note [2, 1]: it proves

$$\text{MLE}[\text{INT}(D)](r) = c \in \mathbb{F}_q$$

for a $2^t \times 2^s$ matrix D of W -bit cells whose *bits* are committed over $\mathbb{F}_2$, by carrying the integer row-fold in the exponent of $\mathbb{K} = \text{GF}(2^{128})$, certified by a lazy GKR grand-product forest, and reading the column combination off in the clear over $\mathbb{F}_q$ (§2 recalls the pipeline). At every measured shape the prover is *forest-dominated*: 84–96% of prove time is the forest and its pre-sumcheck [4].

Applications rarely hold one claim. The motivating pattern — ubiquitous in characteristic-2 arithmetizations, where XOR is free — is a *family* of claims on $\mathbb{F}_2$ -linearly dependent vectors: k claims

$$\text{MLE}[\text{INT}(a_i)](r_i) = c_i, \quad i \in [k],$$

where each claimed vector is a public $\mathbb{F}_2$ -linear combination $a_i = L_i(m_1, \dots, m_j)$, taken bit-position by bit-position, of j committed bit-columns $m_1, \dots, m_j$. The running instance is the XOR triple: $k = 3, j = 2$,

$$a_1 = m_1, \quad a_2 = m_2, \quad a_3 = m_1 \oplus m_2.$$

Today the repository’s virtual-XOR machinery (`prove_mle_eval_mod_q_ligerito_with_virtual_xors`) derives the rows of each a_i for free — the committed code is $\mathbb{F}_2$ -linear, so no new commitment is needed — and shares the commitment and the final recursive opening across claims; but each claim still runs its *own* forest body. Three claims cost three forests: $\sim 3\times$ the dominant phase.

One’s first hope is that the dependence $a_3 = a_1 \oplus a_2$ should make the third claim nearly free. §3 shows why no naive route delivers this: over the integers, $\text{INT}(a \oplus b)$ is *not* determined by $\text{INT}(a)$ and $\text{INT}(b)$ — the missing term is the carry vector $a \wedge b$ — and the other natural tool, a random linear combination (RLC) of the k claims with integer coefficients, spends its soundness bits out of the exponent’s 127-bit injectivity budget.

1.2 The construction, in one paragraph

The construction rests on two observations. First, a *weight-agnosticism* property of the base scheme (Lemma 2.2): nothing in the prover-side soundness chain uses the eq-structure of the row weights — any public weight function with values in $[0, q)$, chunked as usual, works verbatim. Second, reduction mod q of a weight function is *semantically free*: the scheme’s final check is native mod- q arithmetic on the sent folds, so replacing any weight by its reduced representative changes nothing the statement can see (§3.3). Together these open a door that integer-coefficient RLCs

cannot use: draw the batching coefficients $\gamma_1, \dots, \gamma_k$ from $\mathbb{F}_q$ itself, fold them into the weights, and *reduce mod q before lifting into the exponent*. The combined weight function

$$\mathcal{W}_b(m) = \left(\sum_{i \in [k]} \gamma_i w_{i,b} L_i(m) \right) \bmod q \in [0, q), \quad m \in \{0, 1\}^j,$$

is exactly as wide as a single claim's weights — the field-sized randomness costs *zero* exponent bits — and depends on the committed data only through the j bits at each position, so **one** forest whose leaf at each position performs a public 2^j -case select on those bits binds the whole family. The derived vectors a_i never materialise. The leaves are multilinear in the j bits via a subset zeta-transform (Lemma 4.3); the pre-sumcheck grows from one channel to $2^j - 1$ channels; the channels of degree ≥ 2 — which are precisely the carry information — exit as monomial-MLE claims and are discharged by one degree- $(j+1)$ sumcheck over the n -variable cube in the cheap field (§4.5). All residual openings batch into the single ring-switch + recursive-Ligerito opening the base scheme already runs.

1.3 Results

- **The construction** (§4, Protocol 4.6): general j and k , requiring only that all claims share the column point (row points arbitrary). Everything inherited from the base scheme is inherited *verbatim*; the inheritance is audited item by item (Table 1).
- **Security** (§5): completeness is an exact identity chain (Theorem 5.1); soundness adds $1/q$ for the γ -combination — proved exactly, and *tighter* than the k/q working estimate (Lemma 5.2, Remark 5.3) — plus the discharge sumcheck's $(j+1)n/|\mathbb{K}|$ and the batching draws (Theorem 5.5).
- **Special cases** (§6): k claims on the *same* column at different row points collapse to the base protocol with a combined weight vector — one forest, no discharge, implementable against today's API (Corollary 6.1); affine forms (complements, XOR-by-constants) contribute only public constants, recovering the repository's `complement_elision` as the degenerate case (Proposition 6.3).
- **Boundaries** (§7): chunks do not collapse (a conservation law for exponent bandwidth, Remark 7.1); families with distinct column points do not batch (two independent obstructions, Proposition 7.2); the carry channels are conserved, not eliminated (§7.3); case tables scale 2^j .
- **Costs** (§8): a worked estimate against the repository's measured $n = 28/n = 30$ numbers. Headline (est.): the XOR triple at ~ 1.3 – $1.8\times$ one claim versus $3.0\times$ today, one sent fold vector instead of three, verifier within $\sim 1\times$ of the batched baseline.
- **Measured** (§8.1): the same-day prototype lands inside the band — 1.74 – $1.87\times$ one claim at $n = 24$ – 28 versus 2.54 – $3.66\times$ for the batched virtual-XOR path and 3.03 – $3.33\times$ for independent proofs; proofs 26–43% smaller than the batched path. Two model assumptions are validated with sharp teeth: the 2^j -case forest costs $1.3\times$ one claim's at $j = 2$ (A1's band), and the discharge costs 0.29 forest-equivalents *once realised bit-resident* (A2) — the naïve $\mathbb{K}$ -materialised discharge the model ruled out was in fact built first and measured exactly as predicted: it consumed the entire forest saving.

1.4 Status, provenance, attribution

The base scheme (§2) is due to Lev Soukhanov [1]; the exposition and the deployed realisation are [2] and [3]. The family construction of §4 was derived in a working session on 2026-07-26 and recorded, unvetted, in `docs/rlc-family-note-prompt.md`. This note is the vetting pass: every identity, bound, and probability below has been re-derived independently of the session record. The outcome: the construction stands as specified; one soundness budget item improves ($k/q \rightarrow 1/q$, Remark 5.3); one architectural element that the session record presented as a design choice is in fact *forced* (the separate n -variable discharge, Lemma 4.4). The prototype was then built and measured the same day (commits 8fbd155, 4dd5df9, dc697e0, 3fa3011, 8b3ef3e, b0d13f7; measurement harness `examples/rlc_ab.rs`; results in §8.1 and the repository README’s dated notes): it confirms the cost band, validates the two load-bearing modelling assumptions, and surfaced one completeness refinement — vanishing channels must be elided (Proposition 6.4) — now part of the protocol as deployed. As in the parent note, Fiat–Shamir compilation of the composed interactive reduction is inherited, not re-proved (Remark 5.6).

2 The base scheme and the interfaces we use

We recall the deployed protocol only to the depth the construction needs, and we state as lemmas exactly the two properties it relies on. Authoritative descriptions: [2, §9] for the construction, [3] for the deployed pipeline, [5] for the verifier check by check.

2.1 Data, statement, notation

Fix the *binding field* $\mathbb{K} = \text{GF}(2^{128})$ and a public $\alpha \in \mathbb{K}^\times$ that generates $\mathbb{K}^\times$; the generator property is *checked* (`is_generator`, against the factorisation of $2^{128} - 1$), and $\text{ord}(\alpha) = 2^{128} - 1$. Fix an *evaluation modulus* q with $q_{\text{bits}} = \lceil \log_2 q \rceil$ (the reference instantiation is the 100-bit prime $q = 2^{100} - 15$); for the batched construction we additionally require q *prime* (Remark 5.4).

Data is a $2^t \times 2^s$ matrix of W -bit cells, W a power of two; write $w_\nu := \log_2 W$. A *bit position* is a pair (p, c) with $p = (b, \omega)$ a *row-bit index* ($b \in \{0, 1\}^t$ a row, $\omega \in \{0, \dots, W - 1\}$ a word-bit) and $c \in \{0, 1\}^s$ a column. There are $n := t + w_\nu + s$ bit variables. A committed *bit-column* is a vector $m \in \{0, 1\}^{2^{t+w_\nu} \times 2^s}$, entries $m[p, c]$. The integer value of a cell of a bit-column a is $\text{INT}(a)(b, c) := \sum_{\omega < W} 2^\omega a[(b, \omega), c]$, and the claim form is

$$\text{MLE}[\text{INT}(a)](r) = \sum_{b \in \{0, 1\}^t} \sum_{c \in \{0, 1\}^s} \text{eq}(b, r^{\text{row}}) \text{eq}(c, r^{\text{col}}) \text{INT}(a)(b, c) \equiv c \pmod{q}, \quad (1)$$

for a point $r = (r^{\text{row}}, r^{\text{col}}) \in \mathbb{F}_q^t \times \mathbb{F}_q^s$. Throughout, $\text{eq}(x, y) = \prod_i (x_i y_i + (1 - x_i)(1 - y_i))$, and we use freely that $\sum_{b \in \{0, 1\}^t} \text{eq}(b, r) = 1$ for every r .

In this note j always denotes the number of base columns of a family and ω the word-bit index (the repository documents write j for the word-bit index; we rename to avoid the collision). S ranges over subsets of $[j] = \{1, \dots, j\}$, $\mathbf{1}_T \in \{0, 1\}^j$ is the indicator vector of $T \subseteq [j]$, and for bit-columns $M_S := \bigwedge_{i \in S} m_i$ denotes the position-wise AND (with $M_\emptyset :=$ the all-ones column). For a vector v over a Boolean cube, $\tilde{v}$ is its multilinear extension.

2.2 The pipeline

The base protocol for one claim (1) on committed data D ([3]; verifier detail in [5]):

1. **Commit.** The bit-matrix is packed 128 bits per $\mathbb{K}$ -element, RS-encoded and Merkleized (flock [6]); the root is published before any challenge.
2. **Weights and chunking.** The row weights are the reduced representatives $w_b := \text{eq}(b, r^{\text{row}}) \bmod q \in [0, q)$. Because a full-width weight would overflow the exponent budget, each w_b is split into base- 2^{c_w} limbs (“chunks”),

$$c_w := 127 - t - W, \quad L := \lceil q_{\text{bits}}/c_w \rceil, \quad w_b = \sum_{l < L} 2^{c_w l} w_b^{(l)}, \quad w_b^{(l)} \in [0, 2^{c_w}). \quad (2)$$

3. **Forest, per chunk l .** The leaf at bit position $((b, \omega), c)$ is the $\mathbb{K}$ -element

$$\text{leaf}^{(l)}[(b, \omega), c] = \alpha^{2^\omega w_b^{(l)} D[(b, \omega), c]} = 1 + D[(b, \omega), c] \cdot (\alpha^{2^\omega w_b^{(l)}} + 1), \quad (3)$$

affine in the single committed bit (characteristic 2). Per column, $\prod_{(b, \omega)} \text{leaf}^{(l)} = \alpha^{u_c^{(l)}}$ with $u_c^{(l)} = \sum_{b, \omega} 2^\omega w_b^{(l)} D[(b, \omega), c]$. The prover *sends* the folds $u^{(l)} \in \mathbb{Z}^{2^s}$; the verifier recomputes the roots $\alpha^{u_c^{(l)}}$ and absorbs them (the Fiat–Shamir binding), and the lazy *merged* GKR forest reduces all 2^s product trees to a single evaluation claim on the leaf-layer MLE at a shared exit point $(\rho^{(l)}, \xi^{(l)}) \in \mathbb{K}^{t+w_\nu} \times \mathbb{K}^s$.

4. **Pre-sumcheck (“presum”), per chunk.** The exit claim $e^{(l)} = \widetilde{\text{leaf}^{(l)}}(\rho^{(l)}, \xi^{(l)})$ is de-black-boxed: since $\sum_{p, c} \text{eq}(\rho, p) \text{eq}(\xi, c) \cdot 1 = 1$,

$$e^{(l)} + 1 = \sum_p R^{(l)}[p] \cdot m_\xi[p], \quad \text{with} \quad \begin{aligned} R^{(l)}[p] &= \text{eq}(\rho^{(l)}, p) (\alpha^{2^\omega w_b^{(l)}} + 1), \\ m_\xi[p] &= \sum_c \text{eq}(\xi^{(l)}, c) D[p, c], \end{aligned}$$

a degree-2 sumcheck over the $t + w_\nu$ row-bit variables (R is the `q_rowbit` table, m_ξ the ξ -combined rows). Its fresh challenges $r^{*(l)}$ leave the *committed bit-MLE claim* $\widetilde{D}(r^{*(l)}, \xi^{(l)}) = \mu^{(l)}$; the verifier’s only $O(2^t W)$ step is evaluating $\widetilde{R}^{(l)}(r^{*(l)})$.

5. **Opening.** The L residual bit-MLE claims are batched into **one** ring-switch + recursive-Ligerito opening [6, 7].
6. **Read-off and checks.** The verifier checks the per-chunk range bound $u_c^{(l)} < 2^{c_w+t+W}$ for every c, l (`ChunkRange`), and accepts iff

$$\sum_{c \in \{0,1\}^s} \text{eq}(c, r^{\text{col}}) \sum_{l < L} 2^{c_w l} u_c^{(l)} \equiv c \pmod{q}. \quad (4)$$

2.3 The two lemmas the construction stands on

Lemma 2.1 (magnitude and injectivity). *Let the row weights of chunk l take values in $[0, 2^{c_w})$ with $c_w = 127 - t - W$. Then for any committed bits, every per-column fold satisfies $0 \leq u_c^{(l)} < 2^{c_w+t+W} = 2^{127}$; and $x \mapsto \alpha^x$ is injective on $[0, 2^{127})$, so a sent fold $\tilde{u}$ in range with $\alpha^{\tilde{u}} = \alpha^{u_c^{(l)}}$ equals the true fold.*

Proof. $u_c^{(l)} = \sum_b \sum_{\omega < W} 2^\omega (\text{chunk value}) \cdot (\text{bit}) \leq 2^t (2^W - 1)(2^{c_w} - 1) < 2^{t+W+c_w}$. Injectivity: if $\alpha^x = \alpha^y$ with $x, y \in [0, 2^{127})$ then $\text{ord}(\alpha) = 2^{128} - 1$ divides $x - y$, and $|x - y| < 2^{127} < 2^{128} - 1$ forces $x = y$. (This is why the generator check on α is load-bearing.) $\square$

Lemma 2.2 (weight-agnosticism). *The pipeline of §2.2 is sound and complete for any public row-weight vector $w \in [0, q]^{2^t}$ in place of $\text{eq}(\cdot, r^{\text{row}}) \bmod q$, with (4) checked against $\sum_c \text{eq}(c, r^{\text{col}}) \sum_b w_b \text{INT}(D)(b, c) \bmod q$. No component uses the eq-structure of the weights.*

Proof (by inspection of the pipeline). The weights enter exactly four places, each consuming only the values $w_b \in [0, q]$: (i) the chunk decomposition (2) (`chunk_row_weights`), which is defined for any $[0, q]$ vector; (ii) the leaf bases $\alpha^{2^\omega w_b^{(l)}}$ (`chunk_pow2_table`), public powers of public integers; (iii) the presum table $R^{(l)}$ (`row_bit_weights`), again public powers; and (iv) the read-off (4), which is mod- q arithmetic on the sent folds against a verifier-computed target. Lemma 2.1 uses only the chunk width, not the weight provenance. The deployed API already takes the weight vector as an explicit argument (`prove/verify_mle_eval_mod_q_ligerito(..., row_weights_q, ...)`); the eq structure exists only in the caller. $\square$

Two further facts we cite as black boxes, with their errors carried symbolically: the merged-forest GKR reduction is sound with error $\varepsilon_{\text{forest}}$ per chunk (its layer sumchecks over $\mathbb{K}$; [3], tests pin lazy = eager), and the ring-switch + recursive Ligerito opener is a knowledge-sound batched bit-MLE opening with error $\varepsilon_{\text{open}}$ [6, 7, 5]. The presum is a standard degree-2 sumcheck; we account its error explicitly below.

3 Why there is no naive discount

3.1 Carries: XOR claims are genuinely new information

Proposition 3.1. *Position-wise, over $\mathbb{Z}$:*

$$\text{INT}(a \oplus b) = \text{INT}(a) + \text{INT}(b) - 2\text{INT}(a \wedge b), \quad (5)$$

and consequently, at any point r , $\text{MLE}[\text{INT}(a \oplus b)](r) = \text{MLE}[\text{INT}(a)](r) + \text{MLE}[\text{INT}(b)](r) - 2 \text{MLE}[\text{INT}(a \wedge b)](r)$ over $\mathbb{F}_q$. Moreover the pair $(\text{MLE}[\text{INT}(a)](r), \text{MLE}[\text{INT}(b)](r))$ does not determine $\text{MLE}[\text{INT}(a \oplus b)](r)$.

Proof. Per bit, $x \oplus y = x + y - 2xy$ on $\{0, 1\}$; sum with the weights of (1), which are the same on both sides. For non-determination, one instance suffices. Take $W = 1$, $t = 1$, $s = 0$, $q = 5$, $r = r^{\text{row}} = 3$: then $\text{MLE}[\text{INT}(v)](r) = (1 - 3)v_0 + 3v_1 = 3v_0 + 3v_1 \bmod 5$. The pairs $(a, b) = ((0, 1), (0, 1))$ and $(a', b) = ((1, 0), (0, 1))$ give the same evaluations $(3, 3)$, but $a \oplus b = (0, 0)$ evaluates to 0 while $a' \oplus b = (1, 1)$ evaluates to 1. (Equivalently for the AND: $a \wedge b$ evaluates to 3, $a' \wedge b$ to 0 — the two extremes, aligned versus disjoint supports.) $\square$

The two extremes are worth naming, because they are the intuition for everything in §7.3: if $a = b$ then $\text{INT}(a \oplus b) = 0$ while $\text{INT}(a) + \text{INT}(b) = 2\text{INT}(a)$ — the carry term cancels everything; if a and b have disjoint supports then $\text{INT}(a \oplus b) = \text{INT}(a) + \text{INT}(b)$ — the carry term vanishes. Between the extremes lies fresh information: the carry vector $a \wedge b$ is a Hadamard (degree-2) object, exactly what committed-carry architectures pay for explicitly (one committed carry column per modular addition in the char-2 SHA-256 provers, after Binius [8]). A third claim about $a_1 \oplus a_2$ therefore cannot be checked in the clear from c_1, c_2 ; *some* proof work must certify the carries. The question is where that work runs — in the exponent (a forest) or in the cheap field (a sumcheck) — and the whole contribution of this note is moving it from the former to the latter.

3.2 Integer-coefficient RLCs spend the exponent budget

The other naive tool is a random linear combination: draw coefficients γ_i , check the single combined claim $\sum_i \gamma_i \text{MLE}[\text{INT}(a_i)](r_i) \equiv \sum_i \gamma_i c_i$. Over the integers this fails on the budget, in every variant. Recall (Lemma 2.1) that a chunk carries $c_w = 127 - t - W$ weight bits per forest pass, and soundness of the combination needs the γ_i to range over $\geq 2^\lambda$ values with $\lambda \approx q_{\text{bits}}$ (a smaller box gives combination error $2^{-\lambda}$).

Proposition 3.2 (integer-RLC accounting). *Let λ be the bit-size of the integer coefficients, and count forest passes (chunks) for the combined claim.*

- (a) **Combine-then-chunk.** *Chunking the unreduced combined weights $w'_b = \sum_i \gamma_i w_{i,b} < k 2^{\lambda+q_{\text{bits}}}$ needs $L' = \lceil (q_{\text{bits}} + \lambda + \lceil \log_2 k \rceil) / c_w \rceil$ passes. Since $c_w \leq 126 < 2q_{\text{bits}}$ at the reference $q_{\text{bits}} = 100$, this is ≥ 2 passes for every shape — at best a $k \rightarrow 2$ collapse, never $k \rightarrow L = \lceil q_{\text{bits}} / c_w \rceil$.*
- (b) **Per-chunk coefficients.** *Applying γ_i inside a chunk's leaf exponents shrinks the usable chunk width to $c'_w = 127 - t - W - \lambda$: at $\lambda = 100$ this is single digits at the reference splits ($c'_w = 9$ at $n = 28$, $t = 17$, $W = 1$: twelve passes for one combined claim, versus $k = 3$ passes for the unbatched family), and negative — the variant is outright impossible — whenever $t + W > 127 - \lambda$ (all $W = 32$ shapes; large- t splits).*
- (c) **Repetition.** *Replacing one λ -bit draw by ν independent (λ/ν) -bit draws multiplies passes: $\nu \cdot \lceil (q_{\text{bits}} + \lambda/\nu) / c_w \rceil \geq \nu \lceil q_{\text{bits}} / c_w \rceil$, growing in ν — repetitions multiply chunk counts faster than they remove forests.*

Proof. (a) is (2) applied to the wider weights; the bound $w'_b < k 2^{\lambda+q_{\text{bits}}}$ is immediate. (b): the leaf exponent becomes $2^{\gamma_i} w_{i,b}^{(l)} \leq 2^{W-1} 2^{\lambda} 2^{c'_w}$ and Lemma 2.1's proof needs $t + W + \lambda + c'_w \leq 127$. (c): each repetition pays its own $\geq \lceil q_{\text{bits}} / c_w \rceil$ passes. $\square$

3.3 The door: reduction mod q is semantically free

The obstruction in Proposition 3.2 is that integer randomness *adds entropy* to the weight function, and every weight bit must ride through the 127-bit exponent pipe. But the base scheme's semantics is mod- q from end to end: the weights are *already* reduced representatives $\text{eq}(b, r) \bmod q$, and the acceptance check (4) is mod- q arithmetic on the sent folds. Consequently:

Replacing any public weight function $b \mapsto v_b$ by its reduction $b \mapsto v_b \bmod q$ changes the sent folds, but changes no mod- q read-off value: the two weight functions are indistinguishable to the statement. In particular, randomness drawn from $\mathbb{F}_q$ and folded into the weights, then reduced, costs zero exponent bits.

This is the door the construction walks through, and the reason an earlier dismissal of RLC batching was wrong: the dismissal priced the combination at integer width (Proposition 3.2), when the scheme only ever needed mod- q faithfulness. What remains is to make the *combined* weight a function the forest can evaluate — it multiplies different bits per claim — which is the case-collapse of the next section.

4 The construction

4.1 Statement, ordering, and the case-collapsed weight function

The family statement: j committed bit-columns $m_1, \dots, m_j$ (sub-columns of the one committed matrix, as in the virtual-XOR path), k claims $(L_i, r_i, c_i)_{i \in [k]}$ with $L_i : \{0, 1\}^j \rightarrow \{0, 1\}$ public $\mathbb{F}_2$ -linear forms (affine forms in §6.2), points $r_i = (r_i^{\text{row}}, r_i^{\text{col}})$ — **the column point r_i^{col} is required to be shared**; the row points r_i^{row} are arbitrary — and claimed values $c_i \in \mathbb{F}_q$, meaning (1) for $a_i := L_i(m_1, \dots, m_j)$ applied position-wise.

Fiat-Shamir ordering. The commitment root and the complete claim list $(L_i, r_i, c_i)_i$ are absorbed *first*; then the verifier draws $\gamma_1, \dots, \gamma_k \leftarrow \mathbb{F}_q$ and both sides set the combined target

$$T := \sum_{i \in [k]} \gamma_i c_i \pmod{q}.$$

(Optionally $\gamma_1 := 1$; see Remark 5.3.)

Definition 4.1 (case-collapsed weight function). For a row $b \in \{0, 1\}^t$ and a *case vector* $m \in \{0, 1\}^j$, with $w_{i,b} := \text{eq}(b, r_i^{\text{row}}) \pmod{q}$,

$$\mathcal{W}_b(m) := \left(\sum_{i \in [k]} \gamma_i w_{i,b} L_i(m) \right) \pmod{q} \in [0, q),$$

where $L_i(m) \in \{0, 1\}$ is lifted to $\mathbb{F}_q$. Chunk each case value as in (2): $\mathcal{W}_b(m) = \sum_{l < L} 2^{c_w l} \mathcal{W}_b^{(l)}(m)$ with $\mathcal{W}_b^{(l)}(m) \in [0, 2^{c_w})$, the *same* $c_w = 127 - t - W$ and $L = \lceil q_{\text{bits}}/c_w \rceil$ as for one claim. The function is public and verifier-computable ($O(2^t k 2^j)$ $\mathbb{F}_q$ -work; in practice 2^j evaluations of a k -term sum per row).

For $W > 1$ the word-bit weight rides *outside* the reduction, exactly as in the base scheme: bit position (b, ω) in case m carries the exponent $2^\omega \mathcal{W}_b^{(l)}(m)$ in chunk l .

Lemma 4.2 (magnitude, family form). *For arbitrary committed $m_1, \dots, m_j$, the per-column per-chunk fold of the family instance,*

$$u_c^{(l)} := \sum_{b \in \{0, 1\}^t} \sum_{\omega < W} 2^\omega \mathcal{W}_b^{(l)}(m((b, \omega), c)), \quad m(p, c) := (m_1[p, c], \dots, m_j[p, c]),$$

satisfies $0 \leq u_c^{(l)} < 2^{c_w+t+W} = 2^{127}$, and Lemma 2.1's injectivity applies unchanged.

Proof. At every bit position exactly *one* case weight contributes — the one selected by the j committed bits at that position — and every case chunk is $< 2^{c_w}$ by construction. Hence $u_c^{(l)} \leq \sum_b \sum_\omega 2^\omega (2^{c_w} - 1) = 2^t (2^W - 1) (2^{c_w} - 1) < 2^{t+W+c_w}$: the bound of Lemma 2.1, with the same constants, for any committed data. The range check the verifier runs (**ChunkRange**, $u_c^{(l)} < 2^{c_w+t+W}$) carries over verbatim. $\square$

4.2 One forest: leaves as a 2^j -case select

Per chunk l , the family forest has the same shape as the base forest — 2^s product trees over $2^t W$ leaves, merged and lazy — with leaf

$$\text{leaf}^{(l)}[(b, \omega), c] := \alpha^{2^\omega \mathcal{W}_b^{(l)}(m((b, \omega), c))}, \quad (6)$$

a select among the 2^j public values $\alpha^{2^\omega \mathcal{W}_b^{(l)}(m)}$, $m \in \{0,1\}^j$, driven by the j committed bits at the position. The derived vectors a_i never materialise. Everything above the leaf layer — the tree products, the merged GKR layers, the lazy schedules, the root recomputation from the sent folds — is untouched: those layers consume leaf *values* in $\mathbb{K}$ and are indifferent to their provenance.

The leaf is multilinear in the j bits, with coefficients given by a subset zeta-transform:

Lemma 4.3 (case-collapse / zeta-transform). *Let $f : \{0,1\}^j \rightarrow \mathbb{K}$ and let F be its unique multilinear extension. In characteristic 2,*

$$F(m_1, \dots, m_j) = \sum_{S \subseteq [j]} \tau_S \prod_{i \in S} m_i, \quad \tau_S = \sum_{T \subseteq S} f(\mathbf{1}_T).$$

Applied to $f(m) = \alpha^{2^\omega \mathcal{W}_b^{(l)}(m)}$: since the L_i are linear, $\mathcal{W}_b(\mathbf{0}) = 0$, so $\tau_\emptyset = \alpha^0 = 1$ and

$$\text{leaf}^{(l)}[p, c] = 1 + \sum_{\emptyset \neq S \subseteq [j]} \tau_S^{(l)}(p) \prod_{i \in S} m_i[p, c], \quad \tau_S^{(l)}(p) := \sum_{T \subseteq S} \alpha^{2^\omega \mathcal{W}_b^{(l)}(\mathbf{1}_T)} \quad (7)$$

for $p = (b, \omega)$.

Proof. Multilinear interpolation over $\{0,1\}^j$ gives $F(m) = \sum_{v \in \{0,1\}^j} f(v) \prod_i (m_i v_i + (1+m_i)(1+v_i))$ in characteristic 2; expanding the product into the monomial basis yields coefficients $c_S = \sum_{T \subseteq S} (-1)^{|S \setminus T|} f(\mathbf{1}_T)$ (inclusion–exclusion / Möbius inversion on the subset lattice), and $(-1) = 1$ in characteristic 2. Sanity check at $j = 1$: $F(m) = f(0) + m(f(0) + f(1))$, which is (3) when $f(0) = 1$. $\square$

Worked case $j = 2$. Writing $A = \alpha^{2^\omega \mathcal{W}_b^{(l)}(1,0)}$, $B = \alpha^{2^\omega \mathcal{W}_b^{(l)}(0,1)}$, $C = \alpha^{2^\omega \mathcal{W}_b^{(l)}(1,1)}$:

$$\text{leaf} = 1 + m_1(A + 1) + m_2(B + 1) + m_1 m_2(A + B + C + 1),$$

which indeed selects $1, A, B, C$ on the four cases. For the XOR triple ($L_1 = m_1$, $L_2 = m_2$, $L_3 = m_1 \oplus m_2$) the case values are, per row b (all mod q):

$$\mathcal{W}_b(1,0) = \gamma_1 w_{1,b} + \gamma_3 w_{3,b}, \quad \mathcal{W}_b(0,1) = \gamma_2 w_{2,b} + \gamma_3 w_{3,b}, \quad \mathcal{W}_b(1,1) = \gamma_1 w_{1,b} + \gamma_2 w_{2,b},$$

the last because $L_3(1,1) = 0$: at positions where both bits are set, the XOR claim contributes nothing — the carry cancellation of Proposition 3.1, now visible inside the weight table.

4.3 The multi-channel presum

Per chunk, the forest exits with the claim $e^{(l)} = \widetilde{\text{leaf}^{(l)}}(\rho^{(l)}, \xi^{(l)})$. Substituting (7) and separating the constant channel (which sums to 1 exactly as in §2.2):

$$e^{(l)} + 1 = \sum_{\emptyset \neq S \subseteq [j]} \sum_p R_S^{(l)}[p] \cdot m_{S,\xi}^{(l)}[p], \quad \begin{aligned} R_S^{(l)}[p] &:= \text{eq}(\rho^{(l)}, p) \tau_S^{(l)}(p), \\ m_{S,\xi}^{(l)}[p] &:= \sum_c \text{eq}(\xi^{(l)}, c) M_S[p, c], \end{aligned} \quad (8)$$

with $M_S = \bigwedge_{i \in S} m_i$ the monomial (AND) columns. The presum is the same degree-2 sumcheck driver as today, now with $2^j - 1$ channels — one $(R_S, m_{S,\xi})$ pair per non-empty S , each of per-variable degree 2 — over the $t + w_\nu$ row-bit variables (the repository’s multi-degree sumcheck

with one group per channel; the monomial row tables $m_{S,\xi}$ are built like the existing ξ -combined rows, from AND-derived rows). With fresh challenges $r^{*(l)}$, the channel exits are the claims

$$\mu_S^{(l)} := \widetilde{m_{S,\xi}^{(l)}}(r^{*(l)}) = \widetilde{M_S}(\zeta_l), \quad \zeta_l := (r^{*(l)}, \xi^{(l)}) \in \mathbb{K}^{t+w_\nu+s},$$

the verifier computing each $\widetilde{R_S^{(l)}}(r^{*(l)})$ itself (§4.6) and checking the final round against $\sum_S \widetilde{R_S^{(l)}}(r^{*(l)}) \cdot \mu_S^{(l)}$.

For $|S| = 1$ the exit $\widetilde{m_i}(\zeta_l)$ is a committed bit-MLE claim — exactly the object the ring-switch + Ligerito opener consumes, as in the base scheme (the $\mathbb{K}$ -side is $\mathbb{F}_2$ -linear, so sub-column embedding is the same free machinery the virtual-XOR path uses). For $|S| \geq 2$ the exit is a claim about the MLE of a *derived* column M_S that is nowhere committed. These are the carry channels, and they need one more reduction.

A channel whose table $\tau_S^{(l)}$ vanishes *identically* carries nothing ($R_S^{(l)} \equiv 0$) and is **elided**: both parties derive the active channel set per chunk from the public case weights and run only the active channels — presum groups, discharge participation, and the residual-opening set all follow. This is not an optimisation but a completeness requirement of the deployed realisation, and identically-zero channels occur for legitimate degenerate families; see Proposition 6.4.

4.4 Why a separate n -variable discharge is forced

One might hope to run the monomial channels at their native degree inside the presum — integrand $\widetilde{R_S}(x) \prod_{i \in S} \widetilde{m_{i,\xi}}(x)$ over the row variables, exiting directly at committed openings, no new reduction. This is *unsound*, not merely inconvenient:

Lemma 4.4 (partial evaluation does not commute with products). *In general*

$$m_{S,\xi}^{(l)}[p] = \sum_c \text{eq}(\xi, c) \prod_{i \in S} m_i[p, c] \neq \prod_{i \in S} \left(\sum_c \text{eq}(\xi, c) m_i[p, c] \right) = \prod_{i \in S} m_{i,\xi}^{(l)}[p].$$

Consequently, once the forest has bound the column variables to ξ , the monomial row table $m_{S,\xi}$ is not any bounded-degree combination of the singleton row tables over the row variables alone, and its consistency with the committed m_i must be established by a reduction that re-opens the column variables: a sumcheck over all n variables.

Proof. A one-column-variable counterexample: $s = 1$, $S = \{1, 2\}$, $m_1 = m_2 = (0, 1)$ at some fixed p . The left side is $\text{eq}(\xi, 1) = \xi$; the right side is ξ^2 ; these differ for all $\xi \notin \{0, 1\}$, and the presum's ξ is a random $\mathbb{K}$ -point. For the second statement: $m_{S,\xi}$ depends on the joint column data — $|S|$ vectors of 2^s bits per row-bit index — while the singleton tables retain only the $|S|$ column-folded values; the counterexample shows the loss is real already at $s = 1$. $\square$

The session record presented the two-stage structure (degree-2 presum, then a separate discharge) as an architecture; Lemma 4.4 shows it is the *only* completion compatible with the deployed row-variable presum. The alternative — running the presum over all n variables with channels at native degree $|S| + 1$, where cube-products *are* position-wise products — is sound and cost-equivalent (both designs pay one 2^n -scale degree- $(j+1)$ pass; see §8), but it replaces the deployed presum rather than extending it, and it gives up the option of keeping the 2^n -scale pass out of the per-chunk loop.

4.5 The monomial discharge

After all presum exits $\{\mu_S^{(l)}\}$ are absorbed, the verifier draws batching challenges $\eta_{l,S} \leftarrow \mathbb{K}$ for the monomial claims ($|S| \geq 2$), and one *eq-sumcheck over all n variables* proves the batch:

$$\sum_{x \in \{0,1\}^n} \sum_{l < L} \sum_{|S| \geq 2} \eta_{l,S} \text{eq}(\zeta_l, x) \prod_{i \in S} \widetilde{m}_i(x) = \sum_{l, |S| \geq 2} \eta_{l,S} \mu_S^{(l)}, \quad (9)$$

sound because on the cube, products of the multilinear $\widetilde{m}_i$ are the position-wise monomials: $\prod_{i \in S} \widetilde{m}_i(x) = M_S[x]$ for $x \in \{0,1\}^n$, so the left side is $\sum_{l,S} \eta_{l,S} \widetilde{M}_S(\zeta_l)$ when the committed data is used. The integrand has per-variable degree $\leq j+1$ (each term: 1 from the *eq* factor, $\leq j$ from the product). The sumcheck’s fresh point $\chi \in \mathbb{K}^n$ leaves the final check

$$\sum_{l, |S| \geq 2} \eta_{l,S} \text{eq}(\zeta_l, \chi) \prod_{i \in S} \widetilde{m}_i(\chi) \stackrel{?}{=} (\text{last round value}),$$

in which the verifier evaluates each $\text{eq}(\zeta_l, \chi)$ itself ($O(nL)$) and the values $\widetilde{m}_i(\chi)$, $i \in \bigcup_{|S| \geq 2} S$, exit as $\leq j$ committed bit-MLE openings at the single point χ .

All residual openings — the singleton exits $\widetilde{m}_i(\zeta_l)$ and the discharge exits $\widetilde{m}_i(\chi)$, at most $jL + j$ claims at $L + 1$ points — are batched into the *one* ring-switch + recursive Ligerito opening, through the same multi-claim machinery the base scheme already uses for its L chunk claims.

Remark 4.5 (the discharge as a forest leaf layer — the realisation). The deployed $j = 2$ realisation of (9) rests on a complement identity: over $\{0,1\}$ -valued columns, as *polynomials*,

$$\prod_{i \in S} \widetilde{m}_i = \prod_{i \in S} (1 + \neg \widetilde{m}_i), \quad \neg m_i := m_i \oplus \mathbf{1},$$

and $1 + \neg \widetilde{m}_i$ is exactly the base scheme’s leaf-bit-affine shape $1 + (\text{bit stream}) \cdot \tau$ with the *complemented* bit stream and $\tau \equiv 1$. The discharge therefore runs on the repository’s tuned forest-leaf sumcheck driver verbatim — the $\widetilde{m}_i$ tables are never materialised (the driver reads the packed complement bits), the *eq* factor is suffix-tensor-factored (never materialised either), and the per-variable structure splits into two phases, rows then columns, joined by absorbed per-chunk entry sums; the phase-two finals *are* the closing openings $\widetilde{m}_i(\chi)$. Measured at $n = 26$: 17.1 ms per prove ≈ 0.29 forest-equivalents — inside (A2)’s 0.2–0.6 band (§8.1). Two rejected realisations, recorded because both failed exactly as the model warned: a $\mathbb{K}$ -materialised discharge (dense *eq*-combination and monomial tables through a generic multi-degree sumcheck) measured 162.8 ms at $n = 26$ — ≈ 2.1 single-claim-equivalents, consuming the entire forest saving, (A2)’s “multiple forest bodies” verbatim — and a one-group dense *eq*-factored form was *slower* than the generic one because that driver parallelises across groups and the discharge is one group.

4.6 The verifier

Beyond running the sumcheck/forest transcripts, the verifier:

1. recomputes the case weights $\mathcal{W}_b^{(l)}(m)$ (Definition 4.1) and, per chunk and non-empty S , evaluates

$$\widetilde{R}_S^{(l)}(r^{*(l)}) = \sum_p \text{eq}(r^{*(l)}, p) \text{eq}(\rho^{(l)}, p) \tau_S^{(l)}(p),$$

an $O(2^t W)$ dot product per table against the position-wise product tensor $\text{eq}(r^*, \cdot) \odot \text{eq}(\rho, \cdot)$ (which factors per variable, as in the base scheme’s `q_rowbit` step). Building all case powers

costs $O(2^t W 2^j L)$ fixed-base exponentiations, and the $2^j - 1$ tables τ_S follow by the fast zeta transform ($O(j 2^j)$ additions per position). This is the base scheme's only non-succinct step, scaled by $2^j - 1$ (it was one table per chunk);

2. recomputes the roots $\alpha^{u_c^{(l)}}$ from the sent folds and absorbs them; checks every $u_c^{(l)} < 2^{c_w+t+W}$ (**ChunkRange**, unchanged — Lemma 4.2);
3. evaluates $\text{eq}(\zeta_l, \chi)$ and closes the discharge as above;
4. accepts iff the read-off matches the combined target:

$$\sum_{c \in \{0,1\}^s} \text{eq}(c, r^{\text{col}}) \sum_{l < L} 2^{c_w l} u_c^{(l)} \equiv T = \sum_i \gamma_i c_i \pmod{q}. \quad (10)$$

Protocol 4.6 (the family batch, summary).

1. Absorb the commitment root and the claim list $(L_i, r_i, c_i)_{i \in [k]}$ (shared r^{col} enforced); draw $\gamma \leftarrow \mathbb{F}_q^k$; set $T = \sum_i \gamma_i c_i$.
2. Both sides build the case weights $\mathcal{W}_b(m)$ and their chunks (Definition 4.1).
3. Per chunk l : run **one** merged forest with the 2^j -case leaves (6); prover sends the folds $u^{(l)}$; roots recomputed and absorbed; GKR exit $(\rho^{(l)}, \xi^{(l)})$.
4. Per chunk: the multi-channel presum (8) over the *active* channels (identically-zero τ_S elided, Proposition 6.4), exits $\mu_S^{(l)}$ at $\zeta_l = (r^{*(l)}, \xi^{(l)})$.
5. Draw $\eta \leftarrow \mathbb{K}$ per active monomial claim; run the discharge (9), exiting at $\leq j$ openings at χ .
6. Batch all residual openings into one ring-switch + recursive Ligerito opening.
7. Verifier checks: transcripts; $\widetilde{M}_S$ evaluations; range checks; read-off (10) against T .

5 Security

5.1 Completeness

Theorem 5.1 (completeness). *If all k claims hold, the honest prover makes the verifier of Protocol 4.6 accept with probability 1.*

Proof. All transcript checks are exact identities for the honest prover (forest, presum (8) — which is Lemma 4.3 evaluated at the exit point — and discharge (9), whose left side equals $\sum \eta_{i,S} \widetilde{M}_S(\zeta_l) = \sum \eta_{i,S} \mu_S^{(l)}$ on committed data); the range checks hold by Lemma 4.2. For the read-off: the chunks reassemble exactly over $\mathbb{Z}$, $\sum_l 2^{c_w l} u_c^{(l)} = \sum_{b,\omega} 2^\omega \mathcal{W}_b(m((b,\omega), c))$, so, mod q ,

$$\begin{aligned} \sum_c \text{eq}(c, r^{\text{col}}) \sum_l 2^{c_w l} u_c^{(l)} &\equiv \sum_c \text{eq}(c, r^{\text{col}}) \sum_{b,\omega} 2^\omega \sum_i \gamma_i w_{i,b} L_i(m((b,\omega), c)) \\ &= \sum_i \gamma_i \sum_{b,c} w_{i,b} \text{eq}(c, r^{\text{col}}) \sum_\omega 2^\omega a_i[(b,\omega), c] \end{aligned}$$

$$\equiv \sum_i \gamma_i \text{MLE}[\text{INT}(a_i)](r_i) = \sum_i \gamma_i c_i = T,$$

using $L_i(m(p, c)) = a_i[p, c]$ (the definition of a_i), $w_{i,b} \equiv \text{eq}(b, r_i^{\text{row}})$, and the shared column point. $\square$

5.2 The γ -combination

Lemma 5.2 (γ -combination). *Let q be prime and $\delta = (\delta_1, \dots, \delta_k) \in \mathbb{F}_q^k$ be fixed and nonzero, and let $\gamma_1, \dots, \gamma_k$ be drawn independently and uniformly from $\mathbb{F}_q$. Then $\Pr[\sum_i \gamma_i \delta_i = 0] = 1/q$. If instead $\gamma_1 := 1$ and $\gamma_2, \dots, \gamma_k$ are uniform, the probability is 0 when $\delta_2 = \dots = \delta_k = 0$ and $1/q$ otherwise; if $\gamma_i := \gamma^{i-1}$ for one uniform γ , it is at most $(k-1)/q$.*

Proof. Pick i_0 with $\delta_{i_0} \neq 0$ and condition on the other coordinates: $\gamma_{i_0} = -\delta_{i_0}^{-1} \sum_{i \neq i_0} \gamma_i \delta_i$ has exactly one solution in the field $\mathbb{F}_q$, so the conditional probability is exactly $1/q$; averaging preserves it. The pinned- γ_1 case is the same computation when some $\delta_i \neq 0$ for $i \geq 2$, and the equation $\delta_1 = 0$ is unsatisfiable otherwise. The powers case is Schwartz–Zippel on the nonzero univariate $\sum_i \delta_i \gamma^{i-1}$ of degree $\leq k-1$. $\square$

Remark 5.3 (the working estimate k/q was loose). The session record budgeted k/q for this step. The exact cost is $1/q$ for independent draws: a *single* linear form carries all k discrepancies at once, so no union bound over claims is needed. The k/q figure is safe but pays a factor k for nothing; the only variant that genuinely costs more is the powers-of-one- γ compression ($((k-1)/q)$), which buys $k-1$ fewer transcript draws — at $|q| \approx 2^{100}$ either choice is irrelevant in the total budget, and we state the theorem with independent draws.

Remark 5.4 (q prime). The base scheme’s read-off tolerates any characteristic- $\neq 2$ ring. The batched construction uses $\mathbb{F}_q$ as a *field* in Lemma 5.2; for composite q the bound degrades to $1/p_{\min}$ ($p_{\min}$ the smallest prime factor: an adversary can hide all discrepancies in the ideal $(q/p_{\min})$). We therefore require q prime — true of the reference $2^{100} - 15$.

5.3 What changed, and what is inherited

Table 1 is the audit the construction’s soundness reduces to: every row marked *unchanged* is consumed by the proof of Theorem 5.5 as a black box with the base scheme’s guarantee, justified by the cited lemma.

5.4 Soundness

Theorem 5.5 (soundness of the family batch). *Model the commitment as binding with a knowledge-sound batched opening (error $\varepsilon_{\text{open}}$), and let $m_1, \dots, m_j$ be the committed columns it determines. Suppose that for some i^* , $\text{MLE}[\text{INT}(L_{i^*}(m_1, \dots, m_j))](r_{i^*}) \not\equiv c_{i^*} \pmod{q}$. Then the verifier of Protocol 4.6 accepts with probability at most*

$$\frac{1}{q} + L(\varepsilon_{\text{forest}} + \varepsilon_{\text{presum}}) + \varepsilon_{\eta} + \varepsilon_{\text{disch}} + \varepsilon_{\text{open}},$$

$$\varepsilon_{\text{presum}} \leq 2(t + w_{\nu})/|\mathbb{K}|, \quad \varepsilon_{\eta} \leq 1/|\mathbb{K}|, \quad \varepsilon_{\text{disch}} \leq (j+1)n/|\mathbb{K}|,$$

over the verifier’s randomness, where $\varepsilon_{\text{forest}}$ is the base scheme’s per-chunk merged-forest error. (With $|\mathbb{K}| = 2^{128}$ and $q \approx 2^{100}$, the bound is dominated by $1/q$.)

Component	Base scheme	Family scheme — why it survives
Commitment, packing, Merkle	flock RS + Merkle	Unchanged; m_i are sub-columns of the one committed matrix.
Weight values	$\text{eq}(b, r) \bmod q \in [0, q)$	$\mathcal{W}_b(m) \in [0, q)$ per case; admissible by weight-agnosticism (Lemma 2.2).
Chunking	$c_w = 127-t-W$, $L = \lceil q_{\text{bits}}/c_w \rceil$	Identical, applied per case value (Definition 4.1).
Leaf layer	affine in 1 bit (3)	multilinear in j bits (7); Lemma 4.3. $\mathbb{K}$ -valued either way.
Forest above leaves, lazy/merged GKR	product trees + layer sumchecks	Unchanged — consumes leaf values only; same error $\varepsilon_{\text{forest}}$ per chunk.
Sent folds, roots, range check	$u_c^{(l)} < 2^{127}$, α -injectivity	Identical bound and argument (Lemma 4.2); one fold vector for the whole family.
Presum	1 channel, degree 2	2^j-1 channels, each degree 2, same driver (8); same per-round error.
<i>Discharge</i>	—	New: degree- $(j+1)$ eq-sumcheck over n variables (9); forced by Lemma 4.4.
Openings	L claims, one recursive open	$\leq jL + j$ claims at $L+1$ points, same batched opener; error $\varepsilon_{\text{open}}$.
Read-off	against c	against $T = \sum \gamma_i c_i$ (10); Lemma 5.2.
Verifier tables	1 per chunk, $O(2^t W)$	2^j-1 per chunk, same shape (§4.6).

Table 1: The inheritance audit. Only the boldfaced row is a new proof component.

Proof. The γ 's are drawn after the root and all (L_i, r_i, c_i) are absorbed, so the true values $y_i := \text{MLE}[\text{INT}(L_i(m))](r_i)$ and the discrepancies $\delta_i := y_i - c_i$ are fixed before the draw, and $\delta \neq 0$ by hypothesis. By Lemma 5.2, except with probability $1/q$ the combined statement is false: $Y := \sum_i \gamma_i y_i \not\equiv T \pmod{q}$. Condition on this.

It remains to bound the probability that the verifier accepts a false combined statement; we walk the chain back to front, charging each reduction its error once. (i) If every $u_c^{(l)}$ sent equals the true fold of Lemma 4.2 for the committed m , then by the completeness computation (Theorem 5.1, which used only the definition of the true folds) the read-off (10) evaluates to $Y \neq T$ and the verifier rejects. So acceptance requires some sent fold to differ from the true one. (ii) A sent fold in range that differs from the true fold has $\alpha^{\tilde{u}} \neq \alpha^u$ (Lemma 2.1 + Lemma 4.2, both folds in $[0, 2^{127})$); hence the recomputed root differs from the product of the *true* leaves (6). The merged GKR forest then either rejects, or reduces to a *false* leaf-MLE exit claim $e^{(l)} \neq \text{leaf}^{(l)}(\rho^{(l)}, \xi^{(l)})$, except with probability $\varepsilon_{\text{forest}}$ per chunk. (iii) A false exit claim survives the presum only if some exit $\mu_S^{(l)} \neq \widetilde{M_S}(\zeta_l)$ (the true values satisfy (8) with equality — Lemma 4.3 — so if all exits are true the last-round check fails), except with probability $\varepsilon_{\text{presum}} \leq \sum_{\text{rounds}} \deg/|\mathbb{K}| = 2(t + w_\nu)/|\mathbb{K}|$ per chunk. (iv) A false singleton exit is a false committed opening, caught by the opener except with probability $\varepsilon_{\text{open}}$. A false monomial exit makes the discharge's right side of (9) differ from $\sum \eta_{i,S} \widetilde{M_S}(\zeta_l)$, except with probability $\varepsilon_\eta \leq 1/|\mathbb{K}|$ over the η -draw (a nonzero $\mathbb{K}$ -linear form in the η 's, drawn after the μ 's are absorbed); then the discharge sumcheck of the true polynomial $\sum \eta \text{eq}(\zeta_l, x) \prod \widetilde{m_i}(x)$ has a false claimed sum, so it either rejects or ends in a false evaluation at χ , except with probability $\varepsilon_{\text{disch}} \leq (j+1)n/|\mathbb{K}|$; a false final evaluation is a false combination of committed openings ($\text{eq}(\zeta_l, \chi)$ verifier-computed), again caught up to $\varepsilon_{\text{open}}$. Summing the exceptional events over the L chunks and the single discharge and opening gives the stated bound; the opening error is charged once because all residual claims batch into one opening call. $\square$

Remark 5.6 (Fiat–Shamir). As in the parent note, the theorem is about the interactive reduction with ideal subprotocols; non-interactive deployment additionally needs round-by-round soundness of the composed protocol under Fiat–Shamir. The family construction adds two draw sites (γ after the statement; η after the presum exits) and one sumcheck; each is of the standard round-by-round shape, but we have not re-derived the composed statement — the same caveat the base scheme already carries [2, 3].

6 Special cases

6.1 Multi-point batching: one column, many row points

Corollary 6.1 (multi-point batching). *Let all k claims be on the same committed column ($j = 1$, $L_i = \text{id}$): $\text{MLE}[\text{INT}(m_1)](r_i) = c_i$ with shared r^{col} and arbitrary r_i^{row} . Then Protocol 4.6 degenerates to the base protocol run with the combined weight vector*

$$w'_b := \left(\sum_i \gamma_i \text{eq}(b, r_i^{\text{row}}) \right) \bmod q$$

and target $T = \sum_i \gamma_i c_i$: two cases, one channel, no discharge, one forest. Soundness error: the base scheme’s plus $1/q$.

Proof. With $j = 1$ the case function has two values, $\mathcal{W}_b(0) = 0$ and $\mathcal{W}_b(1) = w'_b$, so the leaf (6) is exactly the base leaf (3) with weights w' ; the only non-empty channel is $S = \{1\}$ with $\tau_{\{1\}} = 1 + \alpha^{2^\omega w'_b(l)}$, i.e. the base presum table. Theorems 5.1 and 5.5 apply with the discharge terms vacuous. $\square$

Remark 6.2 (implementable against today’s API). Because the deployed prover and verifier already take the row-weight vector as an explicit argument (Lemma 2.2), Corollary 6.1 needs *no* new proving machinery: a wrapper that (i) enforces the Fiat–Shamir ordering of §4.1, (ii) builds w' and T , and (iii) calls `prove/verify_mle_eval_mod_q_ligerito` verbatim, turns today’s k forests for k row-points into one. This is the free first step of the prototype plan.

6.2 Affine forms, complements, and complement_elision

Let the L_i be affine: $L_i(m) = \ell_i(m) \oplus \epsilon_i$ with ℓ_i linear and $\epsilon_i \in \{0, 1\}$. Definition 4.1 goes through unchanged; what breaks is only $\mathcal{W}_b(\mathbf{0}) = 0$, hence $\tau_\emptyset = 1$, in Lemma 4.3.

Proposition 6.3 (affine forms are constants, not channels). *With affine L_i , (7) holds with the constant term 1 replaced by $\tau_\emptyset^{(l)}(p) = \alpha^{2^\omega \mathcal{W}_b^{(l)}(\mathbf{0})}$, and the presum identity (8) becomes*

$$e^{(l)} = \widehat{R_\emptyset^{(l)}} + \sum_{\emptyset \neq S} \sum_p R_S^{(l)}[p] m_{S,\xi}^{(l)}[p], \quad \widehat{R_\emptyset^{(l)}} := \sum_p \text{eq}(\rho^{(l)}, p) \tau_\emptyset^{(l)}(p),$$

with $\widehat{R_\emptyset^{(l)}}$ verifier-computable by one more $O(2^t W)$ table. The channel structure is otherwise governed by the linear parts only: τ_S depends on the ℓ_i alone for $S \neq \emptyset$, and in particular a form with $\ell_i = 0$ (a constant claim) contributes to no channel: for every $S \ni$ such an inert variable, $\tau_S = \sum_{T \subseteq S} \alpha^{(\text{const})} = 2^{|S|} \alpha^{(\text{const})} = 0$ in characteristic 2.

Proof. $\tau_\emptyset$ is the $T = \emptyset$ term of Lemma 4.3, and the constant-channel sum telescopes to $\widehat{R}_\emptyset$ times $\sum_c \text{eq}(\xi, c) = 1$ as in (8). For $S \neq \emptyset$: $\tau_S = \sum_{T \subseteq S} f(\mathbf{1}_T)$ is the S -th zeta coefficient, which vanishes whenever f does not depend on some variable in S (pair each $T \not\ni i$ with $T \cup \{i\}$; the terms cancel in characteristic 2). An affine shift changes f 's values but not which variables it depends on — that is determined by the ℓ_i . $\square$

The repository's `complement_elision` is the extreme point: a claim on $\neg x = x \oplus \mathbf{1}$ against an existing claim on x at the same row weights needs no forest at all, because the all-ones column has the public evaluation $\text{MLE}[\text{INT}(\mathbf{1})](r) \equiv (2^W - 1)(\sum_b w_b)(\sum_c e_c) \pmod{q}$, so the verifier derives $y_{\neg x} = \text{MLE}[\text{INT}(\mathbf{1})](r) - y_x$ in the clear (the deployed check, at $W = 1$). In the family framework this is the statement that the two claims' *linear parts coincide*, so batching them adds no channel and the affine difference lands entirely in $\widehat{R}_\emptyset$ -type public terms — the construction strictly generalises the elision rather than replacing it: elide what is derivable in the clear, batch the rest.

6.3 Vanishing channels and channel elision

Proposition 6.3 exhibited one way a channel dies — an inert variable. The prototype surfaced a second, *generic* way, and with it a completeness requirement the original specification missed.

Proposition 6.4 (vanishing channels). *Fix a chunk l and a position p , and write $f(m) = \alpha^{2^\omega \mathcal{W}_b^{(l)}(m)}$ for the case function of (6). Then $\tau_S^{(l)}(p)$ is the S -monomial coefficient of the multilinear extension of f (Lemma 4.3), so $\tau_S^{(l)} \equiv 0$ iff that monomial is absent from f at every position. Beyond inert variables (Proposition 6.3), this happens whenever $\mathcal{W}$ factors through a proper $\mathbb{F}_2$ -linear quotient of the case space: if $\mathcal{W}_b(m) = g_b(\sigma(m))$ for a linear surjection $\sigma : \{0, 1\}^j \rightarrow \{0, 1\}^{j'}$, $j' < j$, then $f = h \circ \sigma$ and every τ_S with S not in the image of σ 's monomial support vanishes identically. The motivating instance is a pure-XOR family — every claim on the same combination, e.g. $k = 1$, $j = 2$, $L_1 = m_1 \oplus m_2$ (or several such claims at different row points): then $\mathcal{W}_b(1, 0) = \mathcal{W}_b(0, 1)$ and $\mathcal{W}_b(1, 1) = 0$, so f is a function of $m_1 \oplus m_2$ alone and*

$$\tau_{\{1,2\}} = 1 + \alpha^{2^\omega \mathcal{W}_b^{(l)}(1,0)} + \alpha^{2^\omega \mathcal{W}_b^{(l)}(0,1)} + \alpha^{2^\omega \mathcal{W}_b^{(l)}(1,1)} = 1 + A + A + 1 = 0$$

identically, for every chunk and row. Note $\mathcal{W}$ here depends on both variables — this is not the inert case.

Proof. The first claim is Lemma 4.3 read as a statement about multilinear coefficients. For the quotient claim: a function of $\sigma(m)$ with σ linear has a multilinear expansion supported on the monomials of the functions $m \mapsto$ coordinates of $\sigma(m)$ and their products; concretely, for $\sigma(m) = m_1 \oplus m_2$, any $g(m_1 \oplus m_2)$ has AND-coefficient $g(0) + g(1) + g(1) + g(0) = 0$ in characteristic 2, which is the displayed computation. $\square$

Remark 6.5 (elision is a completeness requirement). The realisation derives each channel's exit $\mu_S^{(l)}$ from the presum's per-channel closing value by dividing by $\widehat{R}_S^{(l)}(r^{*(l)})$; for a vanishing channel that denominator is 0 and the verifier would reject an *honest* proof (the prototype found this as a live failure: a lone XOR claim, and equally a multi-point family on one XOR combination, was rejected with the `RHatZero` error). The fix, now deployed (commit `8b3ef3e`): both parties compute the active channel set per chunk from the public case-weight tables — $\tau_S^{(l)}$ is public data — and elide the zero channels from the presum, the discharge, and the residual-opening set; every base column

must appear in at least one claim (which pins the singleton channels up to negligible-probability γ -collisions, handled as **RHatZero** like any other zero-denominator event). Soundness is untouched — an elided channel contributes $R_S \cdot m_S \equiv 0$ to (8) — and completeness for degenerate families is restored. The pure-XOR family is also a pleasing sanity check of §7.3: a family whose combined weight function has no AND content needs no carry channel, and the construction discovers this by itself.

7 Boundaries and negative results

7.1 Chunks do not collapse: a conservation law

Could a second RLC, across the L chunk weight-sets of one claim, collapse the chunks the way the first collapsed the claims? No:

Remark 7.1 (bandwidth conservation). An exponent pass carries at most $c_w = 127 - t - W$ weight bits (Lemma 2.1 is tight up to one bit: the fold must stay below 2^{127} and the sum over 2^t rows and W word-bits uses the rest). A weight function that is mod- q faithful — distinguishes the q possible targets of the claim it carries — has q_{bits} bits of entropy per row. Hence any scheme in this family runs at least $\lceil q_{\text{bits}}/c_w \rceil = L$ passes per *weight function*, and the only lever is the number of weight functions. The claim-RLC of §4 legitimately compresses k weight functions into one because $\mathbb{F}_q$ -randomness reduced mod q adds no entropy (§3.3); a chunk-RLC gains nothing because $(\sum_l \gamma_l' W_b^{(l)}) \bmod q$ is again a $[0, q)$ -valued weight function — it needs the same L chunks it was trying to remove. The circularity is exact: chunking is the statement that one pass holds c_w bits; no reshuffling of q_{bits} bits of necessary entropy changes $\lceil q_{\text{bits}}/c_w \rceil$. (There is a second, independent failure: the read-off (4) needs the chunks in place-value relation $2^{c_w l}$, and per-chunk targets are not part of the statement — a randomized chunk combination is not even well-posed against T .) This is a heuristic law about *this* pipeline, not a lower bound for all protocols; we state it to mark where the construction’s wins come from — fewer weight functions, never more bits per pass.

7.2 Fully different points do not batch

Proposition 7.2 (distinct column points entangle the tables). *Suppose two claims have $r_1^{\text{col}} \neq r_2^{\text{col}}$. Pushing both into one weight function forces position dependence on the column: $\mathcal{W}_{b,c}(m) = (\sum_i \gamma_i w_{i,b} e_{i,c} L_i(m)) \bmod q$ with $e_{i,c} = \text{eq}(c, r_i^{\text{col}}) \bmod q$. Two independent obstructions follow. (i) Verifier succinctness. The presum tables R_S and the verifier’s final evaluations $\widetilde{R}_S(\cdot)$ become (b, ω, c) -indexed with no row/column tensor split: the $O(2^t W)$ step of §4.6 becomes $O(2^n)$ — the verifier reads the whole cube. (ii) The reduction destroys the rank structure. Unreduced, $\mathcal{W}_{b,c}$ is a sum of k rank-one terms $\gamma_i w_{i,b} \cdot e_{i,c}$ and the leaf would factor as k gathers, $\prod_i \alpha^{\gamma_i w_{i,b} e_{i,c} L_i(m)}$ — but then the exponents are unreduced and Proposition 3.2’s budget failures return (magnitude $k \cdot q$ -headroom, or reduce each term separately and pay $\approx k$ chunks). Reduced, $\alpha^{(\sum_i \dots)} \bmod q$ does not split over i ; the $2^t W$ -sized shared tables that make both prover leaf-gathers and verifier evaluation cheap no longer exist.*

Proof. (i) is immediate from τ_S acquiring a c -index: the factorisation of $\sum_{p,c} \text{eq}(\cdot, p) \text{eq}(\cdot, c) \tau_S(p)$ used in §4.6 required τ_S constant in c . (ii): mod- q reduction is not a homomorphism from $(\mathbb{Z}, +)$ landing in exponents of a fixed α — $\alpha^{(x+y) \bmod q} \neq \alpha^{x \bmod q} \alpha^{y \bmod q}$ in general — so the sum must be materialised per (b, c) before exponentiation. $\square$

Note the symmetry: *same row point, different column points* is already free in the base scheme — the folds u depend only on the row weights, so one forest’s fold vector serves any number of column read-offs (4), one per claim, in the clear. The construction of §4 is the mirror image (different row points, shared column point). What does not exist today is a batching for families mixed in *both* coordinates beyond partitioning into shared-column-point groups; see §10.

7.3 Carries are conserved

It is worth being precise about what the construction did *not* do: it did not make $\text{INT}(a_1 \oplus a_2)$ derivable from the other two claims — Proposition 3.1 forbids that. The information content of the carry vector $M_{\{1,2\}} = a_1 \wedge a_2$ (generally: of every monomial M_S , $|S| \geq 2$) is exactly what the $|S| \geq 2$ presum channels carry, and the discharge (9) is where it is paid for: a degree- $(j+1)$ Hadamard-type argument over the cube. The ledger balances as follows.

- *Committed-carry architectures* (the char-2 SHA provers; Binius [8]) commit the AND/carry columns and prove a bit relation — paying commitment bytes plus a zerocheck, in the cheap field.
- *Today’s F2Z* pays k forests: the carry information is recomputed inside every derived claim’s exponent fold — the most expensive venue available.
- *The family batch* pays one degree- $(j+1)$ sumcheck in $\mathbb{K}$: no new commitment (the monomials are virtual), no forest, and the cost is a small multiple of a linear pass over the cube (§8).

The construction’s gain is a *relocation* of conserved information to the cheapest venue that can certify it, mirrored by the weight-side observation of §4.2 that $L_3(1,1) = 0$ deletes the XOR claim’s contribution exactly where the carry cancels.

7.4 Case tables scale 2^j

The prover’s leaf gathers select among 2^j cases per position, and the verifier builds $2^j - 1$ zeta tables; both are exponential in j . For the motivating instances ($j \leq 4$: 16-case tables, the size the repository’s LUT machinery already runs) this is the cheap regime. For a wide $\mathbb{F}_2$ -linear layer — hundreds of derived vectors over dozens of base columns — one would instead *cluster* the relation matrix into groups of low support ($j_{\text{grp}} \leq 4$, say), batch within groups by this construction, and pay one forest per group. The optimal clustering (minimise $\sum_{\text{grp}} L \cdot \text{forest}(j_{\text{grp}})$ subject to covering the relation set) is a hypergraph partitioning problem we leave open (§10).

8 Cost model

All numbers in this section are **estimates pending the prototype** — the companion plan is `docs/rlc-family-proto-prompt.md` — and the baseline columns are the repository’s measured numbers (Apple M4 16 GB, `-C target-cpu=native`, LTO + `unchecked`, fast profile, medians; README reference tables and the dated 2026-07-26 notes [4]). Reference shape $n = 28$ ($t = 17$, $s = 11$, $W = 1$, $L = 1$, so n here $= t + s$): one-claim prove ≈ 0.55 s, of which forest+presum ≈ 0.50 s and the open phase ≈ 0.05 s; verify 4.4 ms; proof 237.8 KiB (forest-side 50.7 — of which the fold vector u is $2^s \cdot 16$ B = 32 KiB — ring-switch 2.0, Ligerito 181.5). At $n = 30$: prove ≈ 2.35 s after the 2026-07-26 levers, $\gtrsim 94\%$ forest.

Modelling assumptions (each checkable in isolation by the prototype):

- (A1) The family forest costs the base forest times a leaf-round factor. The 2^j -case leaves double the case-table bytes and the interleaved bit streams at $j = 2$ (gather counts unchanged), touching the leaf-adjacent buckets — at $n = 28$ these (`leaf_r1`, `leaf3_r2/r3`, the 16-case rounds, `bitgen`, parts of `build_levels`) are $\approx 45\text{--}55\%$ of the forest — by an assumed $1.1\text{--}1.3\times$: forest factor $\approx 1.05\text{--}1.3\times$. The size-gating lessons (`F2Z_LEAF_A2_FACTORED`, `F2Z_LUT_PRFM`, `F2Z_T4_PRFM`) say the constant is regime-dependent; this is the least certain assumption.
- (A2) The discharge is implemented with bit-resident early rounds (the monomials are ANDs of bit rows; the first rounds are LUT/popcount-style passes, as `F2Z_RS_FAST` does for the ring-switch), so it costs a small number of streaming passes over 2^n bits plus a $\mathbb{K}$ -materialised tail: $\approx 0.2\text{--}0.6$ of one forest body. A naive $\mathbb{K}$ -materialised discharge ($16\text{ B} \cdot 2^n$ tables — 4.3 GB at $n = 28$) would cost *multiple* forest bodies and is out of scope by construction.
- (A3) The presum scales by the channel count ($\times(2^j - 1)$) on tables of size 2^{t+w_ν} — ~ 1 ms-scale at the reference shapes — and the open phase grows by the extra residual claims ($\leq jL + j$ vs. L): a few ms.

$n = 28, W = 1, L = 1$	1 claim (measured)	XOR triple, today	XOR triple, family (est.)
forests	0.50 s	$3 \times 0.50 = 1.50$ s	$(1.05\text{--}1.3) \times 0.50 = 0.53\text{--}0.65$ s
presum(s)	~ 1 ms	~ 3 ms	~ 3 ms
discharge	—	—	0.10–0.30 s (A2)
open (shared)	0.05 s	~ 0.06 s	~ 0.06 s
prove total	0.55 s	≈ 1.56 s ($\approx 2.8\times$)	0.69–1.01 s ($\approx 1.3\text{--}1.8\times$)
sent folds u	32 KiB	96 KiB	32 KiB
proof total	238 KiB	≈ 345 KiB	≈ 255 KiB
verify	4.4 ms	$\approx 9\text{--}11$ ms	$\approx 7\text{--}10$ ms

Table 2: Worked estimate, XOR triple at $n = 28$. “Today” = the batched virtual-XOR path (shared commit and open, one forest body per claim, per-claim u); “family” = Protocol 4.6. Proof-size deltas: the family sends *one* u vector and one forest transcript (≈ 100 KiB vs. today’s triple), and adds two extra presum channels (≈ 3.5 KiB), the discharge transcript (n rounds $\times (j+2)$ $\mathbb{K}$ -elements ≈ 1.9 KiB), and ≈ 3 extra ring-switch blocks (≈ 6 KiB). Verify: the $O(2^t W)$ table step runs $2^j - 1 = 3$ tables instead of 3 separate claims’ one each — a wash — and the discharge verification is $O(n)$ -round, ms-free. At $n = 30$ the same model gives $\approx 6.8\text{ s} \rightarrow 2.9\text{--}4.4\text{ s}$ (est.).

Three honest caveats. First, (A1)’s constant is the same kind of number the 2026-07-26 size-gating sessions showed to flip sign across cache regimes; only the A/B protocol of the repository (alternated in-window pairs, memory-fresh box at $n \geq 28$) will settle it. Second, (A2) assumes engineering that does not exist yet; the discharge’s memory behaviour (bit-resident early rounds, factored eq tensors — $L+1$ of them in the multi-chunk case) is a prerequisite at $n \geq 28$, not an optimisation. Third, the gain degrades gracefully in k and j : for k claims in a j -dimensional family the model predicts $[(1.05\text{--}1.3) + (0.2\text{--}0.6)]$ forest bodies against k , i.e. the break-even is already at $k = 2$ (marginal at the pessimistic end of both bands), and the saving approaches $k\times$ for wide families sharing a small basis.

8.1 Measured: the prototype confirms the band

The prototype was built the same day (Phase 1 eager correctness `8fbd155`; the lazy 2^j -case forest, a pure re-wiring of the deployed `Pair2/T4` kernels with case-power tables, pinned byte-

identical to the eager reference, 4dd5df9; the discharge realisation of Remark 4.5, 3fa3011; channel elision, 8b3ef3e). Measurement harness: `examples/rlc_ab.rs` — the XOR triple against, on *one* commitment and statement, (**single**) one claim through the deployed claims-only virtual-XOR path, (**vx3**) all three through that path’s batched forest, and (**ind3**) three independent proofs. Apple M-series 16 GB, repository protocol (alternated in-window, medians of 5; $n = 28$ of 3); layout $\log_2(\text{cols}) = 2$, $W = 1$, x-tensor split $t' \approx s$, $L = 1$. Prove times in ms (ratios to **single**):

n	single	family (rlc3)	vx3	ind3
22	26.8	21.5 (0.80 $\times$) [†]	37.4 (1.39 $\times$)	69.8 (2.60 $\times$)
24	33.3	58.7 (1.76 $\times$)	84.5 (2.54 $\times$)	110.8 (3.33 $\times$)
26	78.3	146.7 (1.87 $\times$)	234.6 (3.00 $\times$)	239.6 (3.06 $\times$)
28	232.5	405.2 (1.74 $\times$)	851.5 (3.66 $\times$)	707.9 (3.04 $\times$)

The predicted 1.3–1.8 $\times$ -vs-3 $\times$ regime is **confirmed at every forest-dominated shape**; proofs are 135/182/249/341 KB against 169/248/378/602 (**vx3**) and 384/524/719/1000 (**ind3**). Phase decomposition at $n = 26$ (per prove): the one 2^j -case forest costs 59.5 ms against the batched baseline’s 182.7 (3.1 $\times$; $k = 3$ pads to $2^{\lceil \log_2 k \rceil} = 4$ tree-sets) and ≈ 138 for three independent forests — the 2^j -case leaf overhead is $\approx 1.3\times$ one claim’s forest, inside (A1)’s band — and the discharge costs 17.1 ms ≈ 0.29 forest-equivalents, inside (A2)’s band *after* the bit-resident realisation (Remark 4.5 records the two rejected realisations, including the $\mathbb{K}$ -materialised one at 162.8 ms). Three measurement footnotes. [†]The $n = 22$ sub-1 $\times$ entry is the tail-dominated regime — the family’s own one-claim proof costs 19.9 ms, so two *extra* claims cost ~ 1 ms — plus a few ms of *Fiat–Shamir grinding luck* in the baseline: the deployed Ligerito configs grind query/fold proof-of-work whose nonce search length is a deterministic function of the transcript (296,927 blake3 tries for that statement against 165,526 for the family’s, identical on every repetition), so in-window medians cannot average it and small- n comparisons must average over *statements*. Second, the batched baseline’s pad-to-power-of-two makes **vx3** $\approx$ or *worse than* **ind3** at $n \geq 26$ for $k = 3$ — the padded fourth tree-set costs the 1.33 $\times$ its shared tail saves. Third, a lone claim gains nothing from the family API (its single: 33.8 vs 33.3 ms at $n = 24$), and a lone XOR claim is cheaper through the virtual-XOR extraction path (74.5 vs 97.5 ms at $n = 26$: extract-once plus a 1-bit-affine forest beats the 4-case forest): the family’s win is strictly the $k \geq 2$ amortisation this note is about.

Shared-point families (added 2026-07-27). The deployment in which all k points coincide was subsequently specialised and measured. At one point the case-weight table of Definition 4.1 is *rank-1* — $\mathcal{W}_b(m) = (w_b \Gamma(m)) \bmod q$ with $\Gamma(m) = \sum_i \gamma_i L_i(m)$, 2^j values total — the family is capped and canonical (two claims with the same form at one point are the *same* claim; after deduplication $k \leq 2^j - 1$, and the maximal family is the full XOR-closure of the j columns), and the Fiat–Shamir statement absorbs *one* weight vector instead of k (`prove/verify_mle_eval_mod_q_ligerito_rlc_family_shared_point`, a deliberately distinct transcript from the general entry; the general path is byte-stable). The maximal families amortise well past the $k = 4/k = 5$ points above: per claim, $k = 15$ proves at $0.14 \times / 0.28 \times / 0.36 \times$ one claim at $n = 22/24/26$ ($k = 7$: $0.19\text{--}0.41 \times$), running 1.9–2.4 $\times$ faster than the batched baseline and 2.6–5.7 $\times$ faster than independent proofs at $n \leq 26$, with proofs $-77\%/ -92\%$ against the two baselines at $n=26$, $k=15$ (272 vs 1193/3595 KB); the batched baseline’s pad-to-power-of-two is a hard wall at this width ($k=15$ pads to 16 eager tree-sets = 17.2 GB at $n=28$ — unrunnable on the measurement box). Three attribution findings. (i) The rank-1 collapse buys the statement and the API, not prover time: the case-weight build plus case exponentiations are $\leq 0.6\%$ of prove at every j , and the verifier’s $O(2^j 2^{t'})$ case-power step is $\approx 5\%$ of a 5.7–11 ms verify at $n=26$ — the reduction wall $\alpha^{(w_b \Gamma_m) \bmod q} \neq (\alpha^{\Gamma_m})^{w_b}$ stands,

but the per-(row, case) powers it forces were already cheap; the family’s verify win over the baselines ($3.0 \times / 5.1 \times$ at $k=15$, $n=26$) is structural (one forest transcript, one fold vector). (ii) At the maximal $j=4$ family the *discharge* dominates the prove: ≈ 20 ms per active $|S| \geq 2$ channel at $n=26$, linear in channel count ($1/4/11$ channels $\rightarrow 18.6/90/257$ ms), against a forest flat at ≈ 64 ms — sharpening the “riding the forest” problem below into the dominant open lever at full width. (iii) The $j \geq 3$ Dense-JIT lazy forest arm flips profitable at scale (-12% at $n=28$, $k=15$; still -2% slower at $n=24$) — the familiar size-gating. Repository record: the README’s 2026-07-27 shared-point note.

9 Structured taps: index-translated operands

(Added 2026-07-27; the working session of *docs/rlc-structured-taps-prompt.md*, Phase-0 record *docs/rlc-structured-taps-phase0.md*, code commits *29825ba-da1af12*.) This section extends the family question to claims on *index-structured* $\mathbb{F}_2$ -linear images of the committed columns and records what was proved, built, and measured. The outcome is a sharp trichotomy: index translations are free as *weight transforms* (batching to essentially one forest at a shared point), they are cheap *individually* inside XORs (the extraction path, at one forest body per claim, closed by a new translated-eq opening primitive), and they are expensive only when a claim *XOR-mixes differently-translated operands* — where the carry channels of §7.3 multiply and defeat the family’s amortisation.

9.1 Setting and the stream reduction

Committed data: $W=1$ bit-columns whose flat entry index is grouped into 2^g -bit words *along the entry axis*, $p = (k \ll g) \mid j$ (the motivating instance: $g = 5$, 32-bit words). A *tap* applies to a column one op of the form ROT^r (translate the low- g index field, cyclically), SHIFT^r (the same with dropout), and/or off^o (translate the word field by o , dropout below word 0); a claim vector is an XOR of taps, $b = \bigoplus_t \text{op}_t(a_{i_t})$. Equivalently $b = M \cdot a$ for a banded block-rotation $\mathbb{F}_2$ -matrix M ; the tap list is the succinctness-preserving normal form of such an M (an arbitrary M costs the verifier $O(N)$ at the residual closure; translations on contiguous index fields are exactly what stays succinct, by §9.2).

Writing each distinct tap as a *stream* $s = \text{op}(a_i)$, every claim is a per-position XOR of streams, so Protocol 4.6 applies *verbatim* with the streams as the family’s base columns (j_{eff} = the number of distinct streams); completeness (Theorem 5.1) needs only the per-position identity and the shared column point, and the $1/q + \dots$ soundness chain inherits once the residual *stream* openings reduce to committed openings. The active channels are governed by Proposition 6.4’s machinery: each claim’s factor $\alpha^{\gamma_i w_{i,b} L_i(m)}$ is a function of the single bit $L_i(m) = \bigoplus_{u \in \text{supp}_i} m_u$, whose zeta support is $\{\emptyset\}$ and the singletons of supp_i ; multiplying over claims, the active channels are the *unions picking at most one stream per claim* — multiplicative across claims. The pinned $j=2$, $k=6$ instance (two identities, two three-tap single-column rotation convolutions, a cross-column mix, a lossy-SHIFT claim; 13 deduped streams) has ~ 900 such channels monolithically and $27 + 31$ under the two-cluster split $\{b_1, b_3, b_5\} / \{b_2, b_4, b_6\}$ — 45 monomial channels, all of $|S| \leq 3$, so the cascade of Remark 4.5 covers the discharge unchanged.

9.2 Openings: translated eq is a carry matrix product

The presum and cascade exit at stream-MLE claims $\tilde{s}(\zeta)$ over $\mathbb{K}$, which must reduce to committed openings. The change of variables is exact — $\tilde{s}(\zeta) = \sum_{p'} \text{eq}(\zeta, \sigma(p')) [\sigma(p') \text{ valid}] \cdot a_i[p']$ for the

forward index map σ — but the translated weight is *not* an eq tensor:

Lemma 9.1 (translation is not a coordinate permutation). *For $2^g \geq 4$ and $0 < c < 2^g$ with $c \neq 2^{g-1}$, there is no point transform $\zeta \mapsto \zeta'$ with $\text{eq}(\zeta, (y + c) \bmod 2^g) = \text{eq}(\zeta', y)$ for all y : at $g = 2$, $c = 1$ the translated vector violates the rank-one relation $v_{00}v_{11} = v_{01}v_{10}$ that every eq tensor satisfies (it forces $\zeta_1^2 = (1 + \zeta_1)^2$). Index translation involves carries; it is a signed coordinate permutation only for $c \in \{0, 2^{g-1}\}$.*

What replaces the missing transform is a transfer-matrix form:

Proposition 9.2 (translated eq as a carry MPS). *For ν bits, a translation constant δ , and mode $\in \{\text{cyclic}, \text{dropout}\}$, the translated eq $V(y) = \text{eq}_\nu(\rho, y + \delta) \cdot [\text{valid}]$ factors as a matrix product of bond dimension 2 over the bits of y : with per-bit factors $\varphi_i(b) = b\rho_i + (1 + b)(1 + \rho_i)$ and the binary addition carry $c_{i+1} = \text{maj}(y_i, \delta_i, c_i)$,*

$$V(y) = f^\top T_{\nu-1}[y_{\nu-1}] \cdots T_0[y_0] e_0, \quad (T_i[y])_{c \rightarrow c'} = \varphi_i(y \oplus \delta_i \oplus c) [c' = \text{maj}(y, \delta_i, c)],$$

$e_0 = (1, 0)^\top$ and $f = (1, 1)^\top$ (cyclic) or $(1, 0)^\top$ (dropout). Each $T_i[y_i]$ is affine in y_i , so the same formula with $y_i \mapsto \zeta_i \in \mathbb{K}$ is the multilinear extension — an $O(\nu)$ -time closed form. Consequently, over the committed coordinate order the opening weight of a stream splits as $\tilde{K}(v, y) = \sum_\beta A_\beta(v) B_\beta(y)$ over at most three public carry classes $\beta = (\gamma, c_\gamma) \in \{(0, 0)\} \cup \{(1, 0), (1, 1) \text{ if } o > 0\}$ (γ = the word-chain carry at the clear/fold boundary, c_γ = the carry at the 128-bit pack cut; the group chain sits inside the low- g clear coordinates and never reaches the cut, so pure rotations stay rank one), and every class's tables are translated slices of plain eq tables.

Deployment (commit 29825ba): per class one ring-switch message, checked as $\mu = \sum_\beta \sum_v A_\beta(v) s_\beta[v]$; the prover's in-pack marginals and basis fills are sparse walks against the sliced tables (the carries are determined per position, so the classes partition the support); and the verifier's succinct residual closure is a matrix-product-state generalisation of the ring-switch tensor-algebra evaluator at $\leq 2\times$ the plain cost, still $O(m \cdot 128^2)$. No new soundness terms — the split is exact algebra and the opener is basis-agnostic. This primitive alone makes tapped claims first-class in the *extraction* path: extract the claim vector (whole-run gathers; rotations permute clear rows, word offsets shift the folded runs by the borrow), run the usual bit-affine forest, and close the residual's per-tap terms by translated-eq openings.

9.3 Measured: the stream family loses to extraction

Both paths were built and A/B'd on the pinned instance (`examples/taps_ab.rs`; medians of 5, one statement per n ; $n=26$ on a churned box, $n=28$ skipped for memory honesty):

n	single	stream family	batched tap claims	6 independent
22	25.5	257.9 (10.1 $\times$)	118.7 (4.7 $\times$)	151.9 (6.0 $\times$)
24	94.4	743.4 (7.9 $\times$)	319.3 (3.4 $\times$)	401.2 (4.3 $\times$)
26	146.0	3823 (26.2 $\times$)	2006 (13.7 $\times$)	940 (6.4 $\times$)

The family loses at every shape, and the attribution is structural, not incidental: at $n=24$ the 45-channel cascade costs ≈ 600 of 743 ms — ≈ 13 ms per active channel, the *same* per-channel discharge constant the shared-point addendum measured independently — against ≈ 5 saved forest bodies. The instance's sharing is thin (13 streams for 6 claims, one shared tap): the family's economy is *claims per base vector*, exponential for XOR-closures over a fixed small basis ($k \leq 2^j - 1$

over j columns) and inverted here ($k < j_{\text{eff}}$, each claim minting fresh streams, channels multiplying as unions). Carries are conserved (§7.3): claims over a shared basis draw on one carry pool of $\leq 2^j - 1 - j$ monomials; tapped claims each bring their own. What survives for the family: proof size (-20% / -37% vs the batched path at $n=24/26$) and ms-class verify. Routing rule: tapped-convolution claims go through the extraction path unless the claim set has dense stream reuse.

(*Blocked-batching addendum, same day.*) The batched path’s pad was subsequently removed: claims now run in *2-set blocks* through the same batched common (each block its own forest + presum absorbs and exit point, sequential Fiat–Shamir composition; the statement, the ring basis, and the one closing Ligerito call shared). A block-size scan found two tree-sets best-or-tie at every shape — the forest’s marginal round-sharing saturates there, wider merges pay the cache regime — flattening the batched path’s working set to ≈ 2 tree-sets for *any* k . The $k=6$ instance improves to 90.5/250.5/713.1/2441 ms at $n = 22/24/26/28$ ($3.6\text{--}5.8\times$ a single claim): the padded path’s $n=26$ inversion is cured, the batch beats independent proofs at every shape, and $n=28$ runs for the first time. The family comparison above predates the blocking; its verdict is unchanged (the family’s cost is channels, not the baseline’s pad).

9.4 The collapse: uniform-op claims are weight transforms

The positive counterpart, and the reason translations are not intrinsically expensive. For *any* tap op and any extractable x ,

$$\sum_p w[p] \cdot \text{op}(x)[p] = \sum_{p'} w[\sigma(p')] [\text{valid}] \cdot x[p'] : \quad (11)$$

the op moves into the weights, and weight-agnosticism (Lemma 2.2) accepts the transformed vector. Two structural facts make this collapse at a *shared* evaluation point. The group translation acts inside the clear axis ($g \leq s$), so $w \circ \sigma$ keeps the row $\otimes$ column tensor split with the row side unchanged; and the word offset splits into exactly two carry branches, branch 1 using the row weights advanced one folded step (zero at the top — the overflow dropout). Hence k claims, each a *single* tap of a column, γ -combine into at most $(\# \text{columns}) \times 2$ *plain single-column claims* through the deployed claims-only path — no streams, channels, discharge, or translated-eq rings; the verifier derives each (column, branch) value from the proof’s own forest-bound folds and checks their sum against $T = \sum_i \gamma_i c_i$ (error $1/q$ plus the inherited terms; commit **da1af12**). Measured on the instance’s 13 streams as 13 individual claims at one point: 53.5/177.3/603 ms at $n = 22/24/26$ with *four* inner claims, versus 215/609/3674 for the batched tap path (13 claims pad to 16 tree-sets — the pad wall returns at $n=26$) and 347/737/2125 for independent proofs; proofs $-58\text{--}63\%$; verify $\approx 2\times$ faster.

The same argument applies with x an *XOR set*: a claim $\text{op}(a_{i_1} \oplus \dots)$ — e.g. $\text{ROT}^c(a_1 \oplus a_2)$ — is by (11) a standard virtual-XOR claim at transformed weights, and all rotations of one XOR combination at a shared point merge into a single claim (pure column-weight arithmetic; offsets add the one branch). The general law:

$\bigoplus_t \text{op}_t(a_{i_t})$ is cheap whenever the op is common to all terms — it factors out as $\text{op}(\bigoplus_t a_{i_t})$, a weight transform of a plain column-XOR — and expensive precisely when the XOR mixes differently-translated operands, which is where the carry channels live. Rotation, shift, and offset are never the cost; cross-operand XOR-mixing is.

The collapse is deployed for XOR-set sources too (TapPointClaim carries the set; claims $\text{op}(\bigoplus_{i \in \text{cols}} a_i)$ group by (canonical set, branch)). CLI presets for all paths: **f2z <n> --taps**

`vx|family|collapse|rotxor`, the last an 8-claim $\text{ROT}^c(a_1 \oplus a_2)$ -style family collapsing to four inner claims; conventions pending confirmation (transcription fixes and translation direction) are listed in the Phase-0 record.

9.5 The composed collapse: uniform ops over mixed sources

(Added later the same day; commit tag `0x45`, *TapComposedClaim*.) Identity (11) never used that its source is a plain column XOR — only that the inner claim $\langle w \circ \sigma, x \rangle$ is provable. Let the source instead be a fixed *XOR-of-taps combination* $x_S = \bigoplus_{t \in S} \text{op}_t(a_{it})$ — extractable, and provable by the tap-claims path of §9.2 — and take k claims $\text{OUTER}_i(x_{S_i})$ at one shared point $w_{\text{row}} \otimes e$. The branch split of the transformed weight depends only on (OUTER, layout), never on the source, so the γ -combination gives

$$\sum_i \gamma_i \cdot \text{claim}_i = \sum_{(S, \beta)} \langle w_{\text{row}}^{(\beta)} \otimes E_{S, \beta}, x_S \rangle, \quad E_{S, \beta} = \sum_{i: \text{canon}(S_i) = S} \gamma_i e_i^{(\beta)} \bmod q :$$

at most $2 \cdot \#\{\text{distinct sources}\}$ *inner tap claims*, independent of k ; the verifier derives each branch value from the proof’s own forest-bound fold vectors and checks their sum against $\sum_i \gamma_i c_i$ (error $1/q$ plus the inherited terms — the §9.4 chain with “committed column” replaced by “extracted source”). Sources are canonicalised per tap and reduced by char-2 pair cancellation; the outer offset’s envelope ($o < 2^{s-g}$) stands *alone* — it does not compound with the source taps’ offsets, since the transform treats x_S as a black box.

This is the XOR-mixed amortisation the stream family of §9.3 could not deliver: *shift-invariant (schedule-shaped) workloads make every round’s mixed combination a word-offset of one fixed combination*, so k rounds cost *two* inner bodies total — the XOR-mixing price (the translated-eq rings and their carry classes) is paid once per distinct source *shape*, not per claim. Measured (48 claims $\text{off}^t(x)$ of one σ -style source $\text{ROT}^7 a_0 \oplus \text{ROT}^{18} a_0 \oplus \text{SHIFT}^3 a_0 \oplus \text{off}^1 a_1$; claim values computed through the offset-folded extraction route, so every verified rep cross-checks the transform algebra; medians of 5):

n	composed	batched (folds to 64 sets)	48 independent	proofs (cmp/batched)
22	39.2	864.9 (22.1×)	1405.8 (35.9×)	189/1996 KB
24	94.8	3242.3 (34.2×)	2895.7 (30.6×)	284/3602 KB
26	279.7	skipped (pad wall)	7334.8 (26.2×)	448/— KB

Forty-eight XOR-mixed claims land at 1.0–1.9× the cost of *one* (25.5/94.4/146 ms), at the single-proof memory footprint; the batched baseline already inverts against independent proofs at $n=24$ (its 64-set pad). Under the blocked batching of §9.3’s addendum the baseline improves to 736.3/1898.1/5980.8 ms ($n=26$ now runs) and the composed collapse holds $18.8 \times / 21.7 \times / 21.7 \times$ against it, byte-identical (its two inner bodies are one unblocked pair). The routing law of §9.3 refines to: single-tap or uniform-op-of-XOR-set at one point $\rightarrow$ the collapse (§9.4); *many outer ops of few mixed sources* at one point (schedules) $\rightarrow$ the composed collapse; irreducibly distinct XOR-mixed claims $\rightarrow$ the batched extraction path; the stream family only for dense stream reuse. CLI preset: `f2z <n> --taps sched`.

10 Open questions

Open problem 10.1 (clustering wide linear layers). For a large $\mathbb{F}_2$ -linear relation set over many base columns, choose the partition into $j \leq j_{\text{max}}$ -support groups minimising total cost (§7.4).

Structure to exploit: shared basis columns across groups (their singleton openings merge), and affine parts (free, Proposition 6.3). The one-global-point case (§8.1, shared-point addendum) now has measured planner inputs: within a group, open the *full* closure — marginal claims are nearly free ($k=15$ costs 47% more than $k=7$ at $n=26$ for $2.1\times$ the claims) — and prefer wider j over more groups until the discharge’s ≈ 20 ms per active channel times (2^j-1-j) exceeds a fresh group’s flat forest-plus-tail (≈ 150 ms at $n=26$): $j = 4$ groups win at the measured shapes, and discharge kernels/fusion move the crossover further toward wide j . A shared-point family must never route through the padding batched baseline ($2^{\lceil \log_2 k \rceil}$ tree-sets is a memory wall at full closure widths).

Open problem 10.2 (riding the forest). The discharge is one more 2^n -scale streaming pass; the forest’s own bottom layers stream the same bit data. Can the discharge’s early (bit-resident) rounds fuse into the forest’s leaf rounds — one pass, two accumulators — the way pass fusion already merged the fold and message passes (F2Z_EQF_FUSE)? Soundness is unaffected (the transcripts interleave but the challenge order of Protocol 4.6 must be preserved: the discharge’s η ’s depend on all presum exits, so only the *data* passes can fuse, not the rounds themselves — this may cap the fusion at the table-build stage).

Open problem 10.3 (constants of the 2^j -case lazy leaf kernels). (A1) in §8: the 2^j -case select is structurally the existing `LeafTables/Pair2/T4At` family with wider tables and j interleaved bit streams. The size-gated table lessons (factored $\Delta\Delta$ tables, stash/T4 prefetch) predict the constant is a cache-regime function; measure per the repository protocol. *Status: settled for $j = 2$* — the 4-case leaf *is* the deployed `Pair2TauSet` shape keyed by the two columns’ bit streams, the 16-case product level above it *is* `T4Bits`, and the lazy family forest is a table re-wiring with zero kernel changes, byte-identical to the eager reference (`4dd5df9`); measured constant $\approx 1.3\times$ at $n = 26$ (§8.1). For $j \in \{3, 4\}$ the 8/16-case *leaf* rounds remain unbuilt: the deployed path runs eager leaves (a Dense-JIT lazy form over shared case tables was built, pinned byte-identical, and measured *slower* at $n = 24-28$ — it survives as the low-peak-memory arm), and the $j = 3$ discharge now runs as a two-level *cascade* of the leaf-bit form of Remark 4.5: each $|S| \geq 3$ channel factors through a 2-bit AND intermediate ($M_1 M_2 M_3 = (M_1 \wedge M_2) \cdot M_3$, the AND rows already extracted for the presum), whose opening is discharged by a second level at the single point ρ , exiting at committed openings — cascade depth 2 for every $j \leq 4$, no dense tables anywhere. Measured ($k = 4$ on $\{m_1, m_2, m_3, m_1 \oplus m_2 \oplus m_3\}$): the family beats both baselines at $n = 24-28$ (81.9 vs 99.1/146.2 ms at $n = 24$) with 32–74% smaller proofs; the margin is thinner than $j = 2$ ’s because the Dense leaf rounds and the 7-channel presum pay the missing kernels’ price.

Open problem 10.4 (API interplay). Supersede or coexist with the batched and virtual-XOR paths? The family construction requires a shared column point and $\mathbb{F}_2$ -relatedness; the virtual-XOR path requires neither. A claim planner would: elide what is clear-derivable (`complement_elision`, Proposition 6.3), partition the rest by column point, family-batch within parts (Corollary 6.1 when $j = 1$), and fall back to per-claim forests across parts. *Measured inputs to the planner* (§8.1): within a shared-column-point $\mathbb{F}_2$ -related part the family dominates the batched path on both time (21–52%) and size (26–43%) from $k \geq 2$; for $k = 1$ the virtual-XOR path is the right tool (and the batched path’s pad-to-power-of-two makes it worse than independent proofs at $k = 3$, $n \geq 26$ — the planner should never batch-pad small k).

Open problem 10.5 (mixed column points). Does any second randomisation level batch claim families with distinct column points, given Proposition 7.2? The obstruction is joint: verifier succinctness wants a row/column tensor split, mod- q reduction destroys the rank- k split that would restore it. A scheme that re-introduces structure (e.g. column points on a shared small subcube, or a per-column-block RLC with $O(2^{s'})$ verifier tables) is not ruled out.

Open problem 10.6 (zero knowledge). The base scheme is not ZK (the Ligerito opening reveals queried rows), so this is inherited, not introduced. The family construction adds two ZK-relevant surfaces: the sent fold vector u (a deterministic function of the committed data and public weights — already true today) and the discharge transcript. Any future masking of the base scheme must also mask the monomial channels.

Open problem 10.7 (formal composition). Remark 5.6: a round-by-round soundness statement for the composed protocol (forest + multi-channel presum + discharge + batched opening) under Fiat–Shamir, matching the deployed transcript order, has not been written for the base scheme either; the family adds the γ and η sites.

Acknowledgements

The base construction is due to Lev Soukhanov (levs57) [1]; the F2Z realisation, the measurement corpus, and the working session that produced the family construction are recorded in the `f2z-pcs` repository [4, 3].

References

- [1] L. Soukhanov (levs57). *Char 2 fieldswitch*. HackMD note, <https://hackmd.io/@levs57/rkQNcJWzMe>.
- [2] A. Garreta. *A Characteristic-2 Field Switch for $\mathbb{F}_p$ Multilinear Evaluations (the modulo- p approach)*. Companion note, esp. §9 (“A one-round, characteristic-2 Brakedown instantiation”); `char2-fieldswitch-note/main.tex`.
- [3] *F2Z design notes*. `f2z-pcs` repository, `docs/DESIGN.md`.
- [4] *F2Z — an integer-MLE-evaluation PCS over an $\mathbb{F}_2$ commitment*. `f2z-pcs` repository, `README.md` (reference tables; dated optimization notes of 2026-07-25/26).
- [5] *The F2Z verifier for the mod- q opening, with Ligerito as a black box*. `f2z-pcs` repository, `docs/verifier-note/`.
- [6] B. Bünz, R. Rothblum, N. Wang. *Flock: Fast Proving for Batch Boolean Computations*. (The `flock-core` library backs F2Z’s commitment and recursive Ligerito opening.)
- [7] B. E. Diamond, J. Posen. *Polylogarithmic Proofs for Multilinears over Binary Towers* (ring-switching). ePrint 2024/504.
- [8] B. E. Diamond, J. Posen. *Succinct Arguments over Towers of Binary Fields* (Binius; the committed-carry treatment of integer addition in characteristic 2). ePrint 2023/1784.
- [9] J. Thaler. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security, 2022. (Sumcheck and GKR background.)